FreyrSCADA

Embedded Solution

FreyrSCADA Embedded Solution

IEC 60870-5-104 Library

User Manual

Copyright: All rights reserved. None of the information contained in this document may be reproduced or stored in a database or retrieval system or disclosed to others without written authorization by FreyrSCADA Embedded Solution Pvt Ltd.

The information in this document is subject to change without prior notice and should not be construed as a commitment by FreyrSCADA Embedded Solution Pvt Ltd. FreyrSCADA Embedded Solution Pvt Ltd do not assume responsibility for any errors, which may be in this document.

Documentation Control

Author:	Arun
Version:	V 11.29
API Version	V 21.04.011
Creation Date:	14 Aug 13
Last Version Date:	21 Sep. 18
Document Status:	RELEASED

Table of Contents

FreyrSCADA Embedded Solution 1

1 Introduction 5

1.1 Abbreviations 5
1.2 Reference 6

2 Application Programming Interface 7

2.1 IEC104GetLibraryVersion 8
2.2 IEC104GetLibraryBuildTime 9
2.3 IEC104Create 10
2.4 IEC104LoadConfiguration 13
2.5 IEC104Start 23
2.6 IEC104Update 24
2.7 IEC104Read 27
2.8 IEC104Write 28
2.9 IEC104Select 29
2.10 IEC104Operate 31
2.11 IEC104Cancel 33
2.12 IEC104GetDataTypeAndSize 35
2.13 IEC104SetDebugOptions 36
2.14 IEC104ErrorCodeString 37
2.15 IEC104ErrorValueString 38
2.16 IEC104ReadCallback 39
2.17 IEC104WriteCallback 40
2.18 IEC104UpdateCallback 41
2.19 IEC104ControlSelectCallback 43
2.20 IEC104ControlOperateCallback 44
2.21 IEC104ControlPulseEndActTermCallback 45
2.22 IEC104ControlCancelCallback 46
2.23 IEC104DebugMessageCallback 47
2.24 IEC104ControlFreezeCallback 48
2.25 IEC104ParameterActCallback 49
2.26 IEC104DirectoryCallback 50
2.27 IEC104ClientStatusCallback 51
2.28 IEC104ServerStatusCallback 52
2.29 IEC104GetFile 54
2.30 IEC104SendFile 55
2.31 IEC104ClientStatus 56
2.32 IEC104ClientChangeState 57
2.33 GetIEC104ObjectStatus 58
2.34 IEC104ListDirectory 59
2.35 IEC104Stop 60
2.36 IEC104ParameterAct 61
2.37 IEC104Free 63

3 Flow Diagrams 64

3.1 Server 64
3.2 Client 65

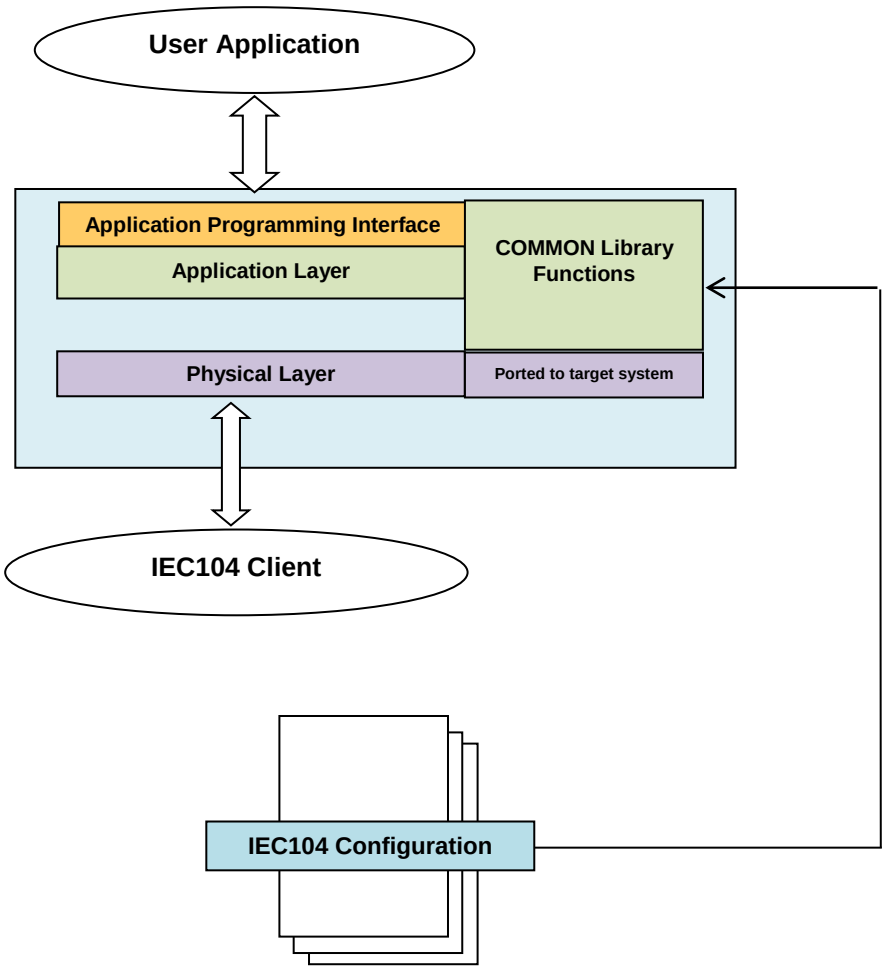
4 Trouble Shooting 66

4.1 Error Codes 66
4.2 Error Values 69

1 Introduction

The IEC104 library acts as interface between user applications and the IEC104 protocol.

The diagram below shows the library software setup and implementation for customer applications.



1.1 Abbreviations

IEC	= International Electro technical Commission.
API	= Application Programming Interface
ID	= Identification
n/a	= not applicable

1.2 Reference

[1] ***Document Title:*** ***IEC 60870-5-104***
Revision: ***Second edition 2006-06***
Author: ***International Electrotechnical Commission***

2 Application Programming Interface

The IEC104 application programming interface (API) consists of a number of function calls used by user application for controlling the IEC104 software stack and call-back functions are provided to get information from the IEC104 stack into the user application

The API consists the following functions

No	Functions	Description
1	IEC104GetLibraryVersion	Get version
2	IEC104GetLibraryBuildTime	Get build date and time
3	IEC104Create	Create object
4	IEC104LoadConfiguration	Load Configuration
5	IEC104Start	Start
6	IEC104Update	Update
7	IEC104Read	Read command
8	IEC104Write	Write command
9	IEC104Select	Select command
10	IEC104Operate	Operate command
11	IEC104Cancel	Cancel command
12	IEC104GetDataTypeAndSize	Get the data type and size
13	IEC104SetDebugOptions	Set Debug Options
14	IEC104ErrorCodeString	Get the Error Code String
15	IEC104ErrorValueString	Get the Error Code String
16	IEC104ReadCallback	Read call-back
17	IEC104WriteCallback	Write call-back
18	IEC104UpdateCallback	Update call-back
19	IEC104ControlSelectCallback	Select call-back
20	IEC104ControlOperateCallback	Operate call-back
21	IEC104ControlPulseEndActTermCallback	Pulse End Act Term call-back
22	IEC104ControlCancelCallback	Cancel call-back
23	IEC104DebugMessageCallback	Debug call-back
24	IEC104ControlFreezeCallback	Freeze Call back
25	IEC104ParameterActCallback	Parameter act Call back
26	IEC104DirectoryCallback	Directory Call back
27	IEC104ClientStatusCallback	Client connection Status Call back
28	IEC104ServerStatusCallback	Server Connection Status Call back
29	IEC104GetFile	Get file function in Monitor direction (File transfer from server to client)
30	IEC104SendFile	Send file function in control direction (File transfer from client to server)
30	IEC104ClientStatus	Get Client connection status
31	IEC104ClientChangeState	Change the state of client test mode /data mode
32	GetIEC104ObjectStatus	IEC104 Object status
33	IEC104ListDirectory	IEC104 Directory read
24	IEC104Stop	Stop Server / Client
35	IEC104Free	Free Server / Client

2.1 IEC104GetLibraryVersion

Syntax

PUBLICAPIPX const Integer8 *PUBLICAPISX IEC104GetLibraryVersion (void)

Parameters

None

Return

Version number of the library as string

Description

The Version number of the library formatted as AA.BB.CCC, where
AA – Major Version Number
BB – Minor Version Number
CCC – Beta/Test Release.

Usage

printf("Version number: %s", IEC104GetLibraryVersion ());

2.2 IEC104GetLibraryBuildTime

Syntax

PUBLICAPIPX const Integer8* PUBLICAPISX IEC104GetLibraryBuildTime (void)

Parameters

None

Return

Build time of the library as a string

Description

The Build time of the library formatted as Mmm dd yyyy hh:mm:ss, where
Mmm – Month
dd – Day
yyyy – Year
hh – Hour
mm – Minutes
ss – Seconds

Usage

printf("Build Time: %s", IEC104GetLibraryBuildTime ());

2.3 IEC104Create

Syntax

PUBLICAPIPX IEC104Object PUBLICAPISX IEC104Create (struct sIEC104Parameters *psParameters, enum eTgtErrorCodes *peErrorCode, tErrorValue *ptErrorValue)

Parameters

Field	Description	
[in]	**psParameters**	IEC 104 Object Parameters
[out]	**peErrorCode**	Pointer to Error Code (if any error occurs)
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

The psParameters is a pointer to structure (struct sIEC104Parameters) containing the following fields

Field	Description
enum eApplicationFlag	Type of Application from one specified below APP_SERVER — Sever Application (Default) APP_CLIENT — Client Application
u32Options	Options used by the application as specified below. Combination of more options can be used by using a Bitwise OR operator APP_OPTION_NONE — No Options (Default) APP_CHECK_TIMESTAMP — When the event is updated check if the time stamp of the event has changed along with the value and status. APP_NEWFRAME_TIMESTAMP_CHANGE — New IEC frame when timestamp change occurs APP_SEQUENCE_BIT — Support frames with sequence bit APP_UPADATEONLY_ON_TIMESTAMP_CHANGE — Create event only if timestamp changes
ptReadCallback	Pointer to the provided application function which will be called when write on an object is performed. Default value is NULL and must be used by Server Only. The read call-back is called when the master sends a read request to the slave (See IEC104ReadCallback)
ptWriteCallback	Pointer to the provided application function which will be called when write on an object is performed. Default value is NULL and must be used by Server Only. The write call-back is called when the master sends a time synchronization message to the slave. (See IEC104WriteCallback)
ptUpdateCallback	Pointer to the provided application function which will be called when object data changes. Default value is NULL and must be used by Client Only. (See IEC104UpdateCallback)
ptSelectCallback	Pointer to the provided application function which will be called when select on an output object is performed. Default value is NULL and must be used by Server Only.

	(See IEC104ControlSelectCallback)
ptOperateCallback	Pointer to the provided application function which will be called when operate on an output object is performed. Default value is NULL and must be used by Server Only. (See IEC104ControlOperateCallback)
ptCancelCallback	This parameter must be NULL as it is not used.(See IEC104ControlCancelCallback)
ptFreezeCallback	Function called when a Freeze Command is executed. If equal to NULL then callback is not used.(See IEC104ControlFreezeCallback)
ptPulseEndActTermCallback	Function called when a pulse command time expires. If equal to NULL then callback is not used .(See IEC104ControlPulseEndActTermCallback)
ptDebugCallback	Pointer to application function which will be called based on options set by sIEC104DebugParameters function or sDebug using the parameter in load configuration function. (See IEC104DebugMessageCallback)
ptParameterActCallback	IEC104ParameterActCallback
ptDirectoryCallback	IEC104DirectoryCallback
ptClientStatusCallback	IEC104ClientStatusCallback
u16ObjectId	Unsigned16 Object identifier
ptServerStatusCallback	IEC104ServerStatusCallback

Return

Pointer to a new IEC104 Object or NULL on error.

Description

Create server or client object

Usage

```
// Sample Server Create

enum eAppErrorCodes          eErrorCode              = APP_ERROR_NONE;
IEC104Object                 myIEC104ObjServer       = NULL;
struct sIEC104Parameters     sParameters             = {0};
tAppErrorValue               tErrorValue             = APP_ERRORVALUE_NONE;

// Setup the required object parameters
sParameters.eAppFlag            = APP_SERVER;              // IEC 104 Server
sParameters.u32Options          = APP_OPTION_NONE;         // No Options
sParameters.ptReadCallback    = cbRead;          // Read Callback
sParameters.ptWriteCallback   = cbWrite;                    // Write Callback
sParameters.ptUpdateCallback  = NULL;          // Update Callback
sParameters.ptSelectCallback  = cbSelect;          // Select Callback
sParameters.ptOperateCallback = cbOperate;          // Operate Callback
sParameters.ptCancelCallback  = cbCancel;          // Cancel Callback
sParameters.ptFreezeCallback  = cbFreeze;          // Cancel Callback
sParameters.ptDebugCallback   = cbDebug;        // Debug Callback
sParameters.ptPulseEndActTermCallback = cbpulseend;          // pulse end callback


// Create a server object
myIEC104ObjServer = IEC104Create(&sParameters, &eErrorCode, &tErrorValue);
if(myIEC104Obj == NULL)
{
        printf("Server Failed to create: %i %i", eErrorCode, tErrorValue);
}

// Sample Client Create
```

```
enum eAppErrorCodes             eErrorCode                  = APP_ERROR_NONE;
IEC104Object                    myIEC104ObjClient           = NULL;
struct sIEC104Parameters        sParameters                 = {0};
tAppErrorValue                  tErrorValue                 = APP_ERRORVALUE_NONE;

// Setup the required object parameters
sParameters.eAppFlag            = APP_CLIENT;               // IEC 104 Server
sParameters.u32Options          = APP_OPTION_NONE;          // No Options
sParameters.ptReadCallback      = NULL;                     // Read Call-back
sParameters.ptWriteCallback     = NULL;                     // Write Call-back
sParameters.ptUpdateCallback    = cbUpdate;                 // Update Call-back
sParameters.ptSelectCallback    = NULL;                     // Select Call-back
sParameters.ptOperateCallback   = NULL;                     // Operate Call-back
sParameters.ptCancelCallback    = NULL;                     // Cancel Call-back
sParameters.ptDebugCallback     = cbDebug;                  // Debug Call-back
sParameters.ptPulseEndActTermCallback = NULL;               // Pulse end callback

// Create a client object
myIEC104ObjClient = IEC104Create(&sParameters, &eErrorCode, &tErrorValue);
if(myIEC104Obj == NULL)
{
        printf("Client Failed to create: %i %i", eErrorCode, tErrorValue);
}
```

2.4 IEC104LoadConfiguration

Syntax

```
enum eAppErrorCodes IEC104LoadConfiguration( IEC104Object myIEC104Obj,
                                             struct sIEC104ConfigurationParameters * psIEC104Config,
                                             tAppErrorValue * ptErrorValue )
```

Parameters

Field	Description
myIEC104Obj	IEC104 Object created using IEC104Create
psIEC104Config	Pointer to a IEC 104 Configuration parameters (See sIEC104ConfigurationParameters)
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

The psIEC104Config is a pointer to structure (struct sIEC104ConfigurationParameters) containing the following fields

Field	Description
sServerSet	Server settings (See struct sServerSettings)
sClientSet	Client settings (See struct sClientSettings)

The Debug parameters (struct sIEC104DebugParameters) contains the following fields

Field	Description
u32DebugOptions	The Debug Options can be Logical OR of items from below. The debug callback will be called based on these settings DEBUG_OPTION_NONE — No Debug Options (Default) DEBUG_OPTION_ERROR — Report Errors DEBUG_OPTION_WARNING — Report Warning DEBUG_OPTION_RX — Report Received Messages DEBUG_OPTION_TX — Report Transmitted Messages

The Server settings (struct sServerSettings) contains the following fields

Field	Description
u8OrginatorAddress	Orginator Address , 0 - not used, 1-255
u16CommonAddress	Common Address of ASDU. The following are the valid values 0 — Not used 1 to 66534 — Station Address 65535 — Global address (master only)
i16k	Maximum difference between receive sequence number to send state variable. Valid values are 1 to 32767 and the default is 12.
i16w	Latest acknowledge after receiving w I format Valid values are 1 to 32767 and the default is 8.
u8t0	Time out of connection establishment in seconds. Valid values are 1 to 255

u8t1	Time out of send or test APDUs in seconds. Valid values are 1 to 255
u8t2	Timeout for acknowledges in case of no data message t2 M t1 in seconds Valid values are 1 to 255.
u8t3	Time out for sending test frames in case of long idle state in seconds. Valid values are 1 to 48
u16ShortPulseTime	Short Pulse Time in milliseconds. Valid values are 0 to 65535
u16LongPulseTime	Long Pulse Time in milliseconds. Valid values are 0 to 65535.
u16EventBufferSize	Buffer Size. Valid values are 100 to 65535
u32ClockSyncPeriod	Clock Synchronization period in milliseconds. If 0 than Clock Synchronization command is not expected from Master
u8MaxAPDUSize	Maximum Length of the APDU. Valid size are 42 to 255. Default value is 255.
bGenerateACTTERMrespond	if Yes , Generate ACTTERM responses for operate commands
sDebug	See struct sIEC104DebugParameters
ai8SourceIPAddress	Server IP Address [MAX_IPV4_ADDRSIZE]
u16PortNumber	Port Number , default port number - 2404
u8MaxNumberofConnection;	max number of parallel client communication
u16NoofObject;	Total number of IEC104 Objects
*psIEC104Objects	See struct sIEC104Object , Pointer to strcuture IEC 104 Objects

The Client settings (struct sClientSettings) contains the following fields

Field	Description
u8TotalNumberofConnection	total number of connection
*psClientConParameters	See struct sClientConnectionParameters, pointer to client connection parameters
sDebug	struct sIEC104DebugParameters , Debug options settings on loading the configuration See struct sIEC104DebugParameters

The Client connection parameters settings (struct sClientConnectionParameters) contains the following fields

Field	Description
u8OrginatorAddress	Orginator Address , 0 - not used, 1-255
u16CommonAddress	Common Address of ASDU. The following are the valid values 0 — Not used 1 to 66534 — Station Address 65535 — Global address (master only)
i16k	Maximum difference between receive sequence number to send state variable. Valid values are 1 to 32767 and the default is 12.
i16w	Latest acknowledge after receiving w I format Valid values are 1 to 32767 and the default is 8.
u8t0	Time out of connection establishment in seconds. Valid values are 1 to 255
u8t1	Time out of send or test APDUs in seconds. Valid values are 1 to 255
u8t2	Timeout for acknowledges in case of no data message t2 M t1 in seconds Valid values are 1 to 255.
u8t3	Time out for sending test frames in case of long idle state in seconds. Valid values are 1 to 48
u16ShortPulseTime	Short Pulse Time in milliseconds. Valid values are 0 to 65535
u16LongPulseTime	Long Pulse Time in milliseconds. Valid values are 0 to 65535.
u32GeneralInterragationInterval	in sec if 0 , gi will not send in particular interval
u32Group1InterragationInterval	in sec if 0 , group 1 interrogation will not send in particular interval
u32Group2InterragationInterval	in sec if 0 , group 2 interrogation will not send in particular interval
u32Group3InterragationInterval	in sec if 0 , group 3 interrogation will not send in particular interval

u32Group4InterragationInterval	in sec if 0 , group 4 interrogation will not send in particular interval
u32Group5InterragationInterval	in sec if 0 , group 5 interrogation will not send in particular interval
u32Group6InterragationInterval	in sec if 0 , group 6 interrogation will not send in particular interval
u32Group7InterragationInterval	in sec if 0 , group 7 interrogation will not send in particular interval
u32Group8InterragationInterval	in sec if 0 , group 8 interrogation will not send in particular interval
u32Group9InterragationInterval	in sec if 0 , group 9 interrogation will not send in particular interval
u32Group10InterragationInterval	in sec if 0 , group 10 interrogation will not send in particular interval
u32Group11InterragationInterval	in sec if 0 , group 11 interrogation will not send in particular interval
u32Group12InterragationInterval	in sec if 0 , group 12 interrogation will not send in particular interval
u32Group13InterragationInterval	in sec if 0 , group 13 interrogation will not send in particular interval
u32Group14InterragationInterval	in sec if 0 , group 14 interrogation will not send in particular interval
u32Group15InterragationInterval	in sec if 0 , group 15 interrogation will not send in particular interval
u32Group16InterragationInterval	in sec if 0 , group 16 interrogation will not send in particular interval
u32CounterInterragationInterval	in sec if 0 , ci will not send in particular interval
u32Group1CounterInterragationInterval	in sec if 0 , group 1 counter interrogation will not send in particular interval
u32Group2CounterInterragationInterval	in sec if 0 , group 2 counter interrogation will not send in particular interval
u32Group3CounterInterragationInterval	in sec if 0 , group 3 counter interrogation will not send in particular interval
u32Group4CounterInterragationInterval	in sec if 0 , group 4 counter interrogation will not send in particular interval
2ClockSyncPeriod	Clock Synchronization period in milliseconds. If 0 than Clock Synchronization command is not expected from Master
u32CommandTimeout	Command timeout, in ms, minimum 3000
bCommandResponseActtermUsed	server sends ACTTERM in command response
ai8DestinationIPAddress	Server IP Address
u16PortNumber	Port Number
u8ReConnection;	Number of Reconnection
u16NoofObject	Total number of IEC104 Objects. Valid values are 1 to 65535
psIEC104Objects	Pointer to the IEC104 Object must match the u8NoofObject parameter. (See struct sIEC104Object)

The IEC104 Object structure (struct sIEC104Object) contains the following fields

Field	Description
eTypeID	Type Identification. Must be one of the following M_SP_NA_1 Single-point information M_SP_TA_1 Single-point information with time tag M_DP_NA_1 Double-point information M_DP_TA_1 Double-point information with time tag M_ST_NA_1 Step position information M_ST_TA_1 Step position information with time tag M_BO_NA_1 Bit string of 32 bit M_BO_TA_1 Bit string of 32 bit with time tag M_ME_NA_1 Measured value, normalized value M_ME_TA_1 Measured value, normalized value with time tag M_ME_NB_1 Measured value, scaled value M_ME_TB_1 Measured value, scaled value with time tag M_ME_NC_1 Measured value, short floating point value M_ME_TC_1 Measured value, short floating point value with time tag M_IT_NA_1 Integrated totals M_IT_TA_1 Integrated totals with time tag M_EP_TA_1 Event of protection equipment with time tag M_EP_TB_1 Packed start events of protection equipment with time tag M_EP_TC_1 Packed output circuit information of protection equipment with time tag M_PS_NA_1 Packed single-point information with status change detection M_ME_ND_1 Measured value, normalized value without quality descriptor M_SP_TB_1 Single-point information with time tag CP56Time2a M_DP_TB_1 Double-point information with time tag CP56Time2a M_ST_TB_1 Step position information with time tag CP56Time2a M_BO_TB_1 Bit string of 32 bit with time tag CP56Time2a M_ME_TD_1 Measured value, normalized value with time tag CP56Time2a M_ME_TE_1 Measured value, scaled value with time tag CP56Time2a M_ME_TF_1 Measured value, short floating point value with time tag CP56Time2a M_IT_TB_1 Integrated totals with time tag CP56Time2a M_EP_TD_1 Event of protection equipment with time tag CP56Time2a M_EP_TE_1 Packed start events of protection equipment with time tag CP56Time2a M_EP_TF_1 Packed output circuit information of protection equipment with time tag CP56Time2a C_SC_NA_1 Single command C_DC_NA_1 Double command C_RC_NA_1 Regulating step command C_SE_NA_1 Set point command, normalized value C_SE_NB_1 Set point command, scaled value C_SE_NC_1 Set point command, short floating point value C_BO_NA_1 Bit string of 32 bit command C_SC_TA_1 Single command with time tag CP56Time2a C_DC_TA_1 Double command with time tag CP56Time2a

	C_RC_TA_1 — Regulating step command with time tag CP56Time2a C_SE_TA_1 — Set point command, normalized value with time tag CP56Time2a C_SE_TB_1 — Set point command, scaled value with time tag CP56Time2a C_SE_TC_1 — Set point command, short floating point value with time tag CP56Time2a C_BO_TA_1 — Bit string of 32 bit command with time tag CP56Time2a C_CI_NA_1 — Counter interrogation command
u32IOA	Information Object Address
u16Range	No of Sequential objects
eIntroCOT	Interrogation group INROGEN — Interrogated by station interrogation INRO1 — Interrogated by group 1 interrogation INRO2 — Interrogated by group 2 interrogation INRO3 — Interrogated by group 3 interrogation INRO4 — Interrogated by group 4 interrogation INRO5 — Interrogated by group 5 interrogation INRO6 — Interrogated by group 6 interrogation INRO7 — Interrogated by group 7 interrogation INRO8 — Interrogated by group 8 interrogation INRO9 — Interrogated by group 9 interrogation INRO10 — Interrogated by group 10 interrogation INRO11 — Interrogated by group 11 interrogation INRO12 — Interrogated by group 12 interrogation INRO13 — Interrogated by group 13 interrogation INRO14 — Interrogated by group 14 interrogation INRO15 — Interrogated by group 15 interrogation INRO16 — Interrogated by group 16 interrogation REQCOGEN — Requested by general counter request REQCO1 — Requested by group 1 counter request REQCO2 — Requested by group 2 counter request REQCO3 — Requested by group 3 counter request REQCO4 — Requested by group 4 counter request
u32CyclicTransTime	Periodic or Cyclic Transmission time in milliseconds. If 0 do not transmit Periodically. Valid values are 0 to 65000
eControlModel	Control Model valid for Output Objects only STATUS_ONLY — Status only DIRECT_OPERATION — Direct Operate SELECT_BEFORE_OPERATION — Select Before Operate
u32SBOTimeOut	Timeout between Select and Operate in milliseconds in the range of 0 to 4294967296
ai8Name[APP_OBJNAMESIZE]	Name of the IEC104 Object (max is 48 characters)

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Loads the IEC104 configuration into the specified server or client object.

Usage

```
// Sample Server Load Configuration
   enum eAppErrorCodes                eErrorCode      = APP_ERROR_NONE;
   tAppErrorValue                     tErrorValue     = APP_ERRORVALUE_NONE;
   struct sIEC104ConfigurationParameters   sIEC104Config   = {0};

         sIEC104Config.sServerSet.u16MaxNumberofConnection          =         1;
         sIEC104Config.sServerSet.psServerConParameters = NULL;
         sIEC104Config.sServerSet.psServerConParameters = calloc(
sIEC104Config.sServerSet.u16MaxNumberofConnection, sizeof(struct sServerConnectionParameters));
         if( sIEC104Config.sServerSet.psServerConParameters == NULL)
         {
                  printf("\r\nError: Not enough memory to alloc objects");
                  break;
         }
         sIEC104Config.sServerSet.psServerConParameters[0].i16k        =         12;
         sIEC104Config.sServerSet.psServerConParameters[0].i16w        =         8;
         sIEC104Config.sServerSet.psServerConParameters[0].u8t0        = 30;
         sIEC104Config.sServerSet.psServerConParameters[0].u8t1        = 15;
         sIEC104Config.sServerSet.psServerConParameters[0].u8t2        = 10;
         sIEC104Config.sServerSet.psServerConParameters[0].u16t3       = 20;
         sIEC104Config.sServerSet.psServerConParameters[0].u16EventBufferSize        =         50;
         sIEC104Config.sServerSet.psServerConParameters[0].u32ClockSyncPeriod                =         0;
         sIEC104Config.sServerSet.psServerConParameters[0].bGenerateACTTERMrespond      =         TRUE;
         strcpy((char*)sIEC104Config.sServerSet.psServerConParameters[0].ai8SourceIPAddress,"127.0.0.1");
         strcpy((char*)sIEC104Config.sServerSet.psServerConParameters[0].ai8RemoteIPAddress,"0.0.0.0");
         sIEC104Config.sServerSet.psServerConParameters[0].u16PortNumber                     =         2404;
         sIEC104Config.sServerSet.psServerConParameters[0].bEnableRedundancy       =         FALSE;
         strcpy((char*)sIEC104Config.sServerSet.psServerConParameters[0].ai8RedundSourceIPAddress,"127.0.
0.1");
         strcpy((char*)sIEC104Config.sServerSet.psServerConParameters[0].ai8RedundRemoteIPAddress,"0.0.0.
0");
         sIEC104Config.sServerSet.psServerConParameters[0].u16RedundPortNumber   =         2400;


         //sIEC104Config.sServerSet.sDebug.u32DebugOptions     =       0;
         sIEC104Config.sServerSet.sDebug.u32DebugOptions     =   ((DEBUG_OPTION_ERROR |
DEBUG_OPTION_TX) | DEBUG_OPTION_RX);
         sIEC104Config.sServerSet.bEnablefileftransfer   = FALSE;
         strcpy((char*)sIEC104Config.sServerSet.ai8FileTransferDirPath, (char*)"C:\\kr\\FileTransferServer");
         sIEC104Config.sServerSet.u16MaxFilesInDirectory                     =         10;
         sIEC104Config.sServerSet.bEnableDoubleTransmission = FALSE;
         sIEC104Config.sServerSet.u8TotalNumberofStations      =         1;
         sIEC104Config.sServerSet.au16CommonAddress[0]      =         1;
         sIEC104Config.sServerSet.au16CommonAddress[1]      =         0;
         sIEC104Config.sServerSet.au16CommonAddress[2]      =         0;
         sIEC104Config.sServerSet.au16CommonAddress[3]      =         0;
         sIEC104Config.sServerSet.au16CommonAddress[4]      =         0;
         sIEC104Config.sServerSet.benabaleUTCtime =  FALSE;

         sIEC104Config.sServerSet.bTransmitSpontMeasuredValue = TRUE;
         sIEC104Config.sServerSet.bTranmitInterrogationMeasuredValue =TRUE;
         sIEC104Config.sServerSet.bTransmitBackScanMeasuredValue = TRUE;

         sIEC104Config.sServerSet.u16ShortPulseTime                          =         5000;
         sIEC104Config.sServerSet.u16LongPulseTime                           =         20000;
```

```
	sIEC104Config.sServerSet.u16NoofObject 		=	2;	// Define number of objects

	// Allocate memory for objects
	sIEC104Config.sServerSet.psIEC104Objects = calloc(	sIEC104Config.sServerSet.u16NoofObject, sizeof(struct sIEC104Object));
	if(	sIEC104Config.sServerSet.psIEC104Objects == NULL)
	{
		printf("\r\nError: Not enough memory to alloc objects");
		break;
	}

	// Init objects
	//first object detail

	strncpy((char*)	sIEC104Config.sServerSet.psIEC104Objects[0].ai8Name,"M_ME_TF_1 100-109",APP_OBJNAMESIZE);
	sIEC104Config.sServerSet.psIEC104Objects[0].eTypeID		=	M_ME_TF_1;
	sIEC104Config.sServerSet.psIEC104Objects[0].u32IOA		=	100;
	sIEC104Config.sServerSet.psIEC104Objects[0].u16Range		=	10;
	sIEC104Config.sServerSet.psIEC104Objects[0].eIntroCOT		=	INRO6;
	sIEC104Config.sServerSet.psIEC104Objects[0].eControlModel	=	STATUS_ONLY;
	sIEC104Config.sServerSet.psIEC104Objects[0].u32SBOTimeOut	=	0;
	sIEC104Config.sServerSet.psIEC104Objects[0].u16CommonAddress	=	1;


	strncpy((char*)sIEC104Config.sServerSet.psIEC104Objects[1].ai8Name,"C_SE_TC_1",APP_OBJNAMESIZE);
	sIEC104Config.sServerSet.psIEC104Objects[1].eTypeID		=	C_SE_TC_1;
	sIEC104Config.sServerSet.psIEC104Objects[1].u32IOA		=	100;
	sIEC104Config.sServerSet.psIEC104Objects[1].eIntroCOT		=	NOTUSED;
	sIEC104Config.sServerSet.psIEC104Objects[1].u16Range		=	10;
	sIEC104Config.sServerSet.psIEC104Objects[1].eControlModel	=	DIRECT_OPERATE;
	sIEC104Config.sServerSet.psIEC104Objects[1].u32SBOTimeOut	=	0;
	sIEC104Config.sServerSet.psIEC104Objects[1].u16CommonAddress	=	1;

	// Load configuration
	eErrorCode = IEC104LoadConfiguration(myServer, &sIEC104Config, &tErrorValue);
	if(eErrorCode != EC_NONE)
	{
		printf("\r\nError: IEC104LoadConfiguration() failed: %d - %s, %d - %s ", eErrorCode, errorcodestring(eErrorCode),	tErrorValue , errorvaluestring(tErrorValue));
	}


// Sample Client Load Configuration
	enum eAppErrorCodes		eErrorCode	= APP_ERROR_NONE;
	tAppErrorValue		tErrorValue	= APP_ERRORVALUE_NONE;
	struct sIEC104ConfigurationParameters	sIEC104Config	= {0};

	sIEC104Config.sClientSet.benabaleUTCtime	=	FALSE;
	 //sIEC104Config.sClientSet.sDebug.u32DebugOptions	=	(DEBUG_OPTION_ERROR | DEBUG_OPTION_WARNING) ;
	 sIEC104Config.sClientSet.sDebug.u32DebugOptions =	( DEBUG_OPTION_TX | DEBUG_OPTION_RX);
	 sIEC104Config.sClientSet.u16TotalNumberofConnection =	1;
		sIEC104Config.sClientSet.psClientConParameters	=	malloc (sIEC104Config.sClientSet.u16TotalNumberofConnection * sizeof(struct sClientConnectionParameters));
```

```
//server 1 config Starts
sIEC104Config.sClientSet.psClientConParameters[0].eState = DATA_MODE;
sIEC104Config.sClientSet.psClientConParameters[0].u8TotalNumberofStations = 1;
sIEC104Config.sClientSet.psClientConParameters[0].au16CommonAddress[0] = 1;
sIEC104Config.sClientSet.psClientConParameters[0].au16CommonAddress[1] = 0;
sIEC104Config.sClientSet.psClientConParameters[0].au16CommonAddress[2] = 0;
sIEC104Config.sClientSet.psClientConParameters[0].au16CommonAddress[3] = 0;
sIEC104Config.sClientSet.psClientConParameters[0].au16CommonAddress[4] = 0;
sIEC104Config.sClientSet.psClientConParameters[0].u8OrginatorAddress = 0;
sIEC104Config.sClientSet.psClientConParameters[0].i16k =
12;
sIEC104Config.sClientSet.psClientConParameters[0].i16w =
8;
sIEC104Config.sClientSet.psClientConParameters[0].u8t0 = 30;
sIEC104Config.sClientSet.psClientConParameters[0].u8t1 = 15;
sIEC104Config.sClientSet.psClientConParameters[0].u8t2 = 10;
sIEC104Config.sClientSet.psClientConParameters[0].u16t3 = 20;

sIEC104Config.sClientSet.psClientConParameters[0].u32GeneralInterrogationInterval = 0; //
in sec if 0 , gi will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group1InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group2InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group3InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group4InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group5InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group6InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group7InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group8InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group9InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group10InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group11InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group12InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group13InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group14InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group15InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32Group16InterrogationInterval = 0; //
in sec if 0 , group 1 interrogation will not send in particular interval
sIEC104Config.sClientSet.psClientConParameters[0].u32CounterInterrogationInterval = 0; //
in sec if 0 , ci will not send in particular interval

sIEC104Config.sClientSet.psClientConParameters[0].u32Group1CounterInterrogationInterval = 0; // in sec if
0 , group 1 counter interrogation will not send in particular interval

sIEC104Config.sClientSet.psClientConParameters[0].u32Group2CounterInterrogationInterval = 0; // in sec if
0 , group 1 counter interrogation will not send in particular interval
```

```
sIEC104Config.sClientSet.psClientConParameters[0].u32Group3CounterInterrogationInterval =     0;    // in sec if
0 , group 1 counter interrogation will not send in particular interval

sIEC104Config.sClientSet.psClientConParameters[0].u32Group4CounterInterrogationInterval =     0;    // in sec if
0 , group 1 counter interrogation will not send in particular interval
        sIEC104Config.sClientSet.psClientConParameters[0].u32ClockSyncInterval  =     0;
         // in sec if 0 , clock sync, will not send in particular interval

         sIEC104Config.sClientSet.psClientConParameters[0].u32CommandTimeout =     10000;
         sIEC104Config.sClientSet.psClientConParameters[0].u32FileTransferTimeout     =          50000;

sIEC104Config.sClientSet.psClientConParameters[0].bCommandResponseActtermUsed  =          TRUE;


strcpy((char*)sIEC104Config.sClientSet.psClientConParameters[0].ai8DestinationIPAddress,"127.0.0.1");
 sIEC104Config.sClientSet.psClientConParameters[0].u16PortNumber                          =          2404;


         sIEC104Config.sClientSet.psClientConParameters[0].bEnablefileftransfer = FALSE;

strcpy((char*)sIEC104Config.sClientSet.psClientConParameters[0].ai8FileTransferDirPath,"C:\\");

sIEC104Config.sClientSet.psClientConParameters[0].bUpdateCallbackCheckTimestamp = FALSE;


 sIEC104Config.sClientSet.psClientConParameters[0].u16NoofObject                          =          2;          //
Define number of objects

                                // Allocate memory for objects
         sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects = calloc(
sIEC104Config.sClientSet.psClientConParameters[0].u16NoofObject, sizeof(struct sIEC104Object));
                                if(   sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects == NULL)
                                {
                                        printf("\r\nError: Not enough memory to alloc objects");
                                        break;
                                }

                                // Init objects
                                //first object detail

strcpy((char*)sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[0].ai8Name,"100");
sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[0].eTypeID                          =
M_ME_TF_1;
 sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[0].u32IOA                          = 100;
 sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[0].eIntroCOT       = INRO1;
 sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[0].u16Range                          = 10;
 sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[0].eControlModel  =
        STATUS_ONLY ;

sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[0].u32SBOTimeOut  = 0;

sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[0].u16CommonAddress    =          1;


strncpy((char*)sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[1].ai8Name,"C_SE_TC_1"
,APP_OBJNAMESIZE);
sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[1].eTypeID                          = C_SE_TC_1;
 sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[1].u32IOA                          = 100;
 sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[1].eIntroCOT       = NOTUSED;
```

```
 sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[1].u16Range                    = 10;

sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[1].eControlModel  = DIRECT_OPERATE;

sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[1].u32SBOTimeOut  = 0;

sIEC104Config.sClientSet.psClientConParameters[0].psIEC104Objects[1].u16CommonAddress    =          1;
                                    // server 1 config ends


 // Load configuration
 eErrorCode = IEC104LoadConfiguration(myClient, &sIEC104Config, &tErrorValue);
 if(eErrorCode != EC_NONE)
 {
printf("\r\nError: IEC104LoadConfiguration() failed:          %d - %s, %d - %s ", eErrorCode,
errorcodestring(eErrorCode),  tErrorValue , errorvaluestring(tErrorValue));

 }
```

2.5 IEC104Start

Syntax

eAppErrorCodes IEC104Start(IEC104Object myIEC104Obj, tAppErrorValue * ptErrorValue)

Parameters

Field	Description
myIEC104Obj	IEC104 Object created by IEC104Create
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Start the IEC104 communication

Usage

```
enum eAppErrorCodes  eErrorCode      = APP_ERROR_NONE;
tAppErrorValue       tErrorValue     = APP_ERRORVALUE_NONE;

//Start the IEC 104 Server or Client Object based on the object created
eErrorCode = IEC104Start(myIEC104Object, &tErrorValue);
if(eErrorCode != APP_ERROR_NONE)
{
        printf("Start IEC 104 has failed: %i %i", eErrorCode, tErrorValue);
}
```

2.6 IEC104Update

Syntax

PUBLICAPIPX enum eTgtErrorCodes PUBLICAPISX
IEC104Update (IEC104Object myIEC104Obj, Boolean bGenEvent, struct sIEC104DataAttributeID *psDAID, struct sIEC104DataAttributeData *psNewValue, Unsigned16 u16Count, tErrorValue *ptErrorValue)

Parameters

Field	Description
myIEC104Obj	IEC104 Object created by IEC104Create
bGenEvent	Generate event, otherwise just update the database value
psDAID	Pointer to data attribute ID (See struct sIEC104DataAttributeID)
psNewValue	Pointer to data attribute data (See struct sIEC104DataAttributeData)
u16Count	Number of points to be updated. Valid values are 1 to 65535.
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

The psDAID is a pointer to structure (struct sIEC104DataAttributeID) containing the following fields

Field	Description
U32IOA	Information Object Number
eTypeID	Type Identification of the Object (See eTypeID)
pvUserData	This parameter must be NULL as it is not used.

The psNewValue is a pointer to structure (struct sIEC104DataAttributeData) containing the following fields

Field	Description
sTimeStamp	The time stamp of the updated IEC104 object. (See struct sSETTimeStamp)
tQuality	The quality of the object. This value could be Bitwise OR of following GD Good IV Invalid NT Not Topical SB Substituted BL Blocked OV Overflow EI Elapsed time invalid TR Equipment in transient state CA Counter was adjusted CR Carry
eDataType	The data type of the object. It must be one of the following UNSUPPORTED_DATATYPE Not supported SINGLE_POINT_DATATYPE 1 Bit Data DOUBLE_POINT_DATATYPE 2 Bit Data UNSIGNED_BYTE_DATATYPE 8 Bit Unsigned Data SIGNED_BYTE_DATATYPE 8 Bit Signed Data UNSIGNED_WORD_DATATYPE 16 Bit Unsigned Data SIGNED_WORD_DATATYPE 16 Bit Signed Data UNSIGNED_DWORD_DATATYPE 32 Bit Unsigned Data SIGNED_DWORD_DATATYPE 32 Bit Signed Data FLOAT32_DATATYPE 32 Bit Float Data

eDataSize	The data type of the object. It must be one of the following UNSUPPORTED_SIZE — Not supported SINGLE_POINT_SIZE — 1 Bit Data Size DOUBLE_POINT_SIZE — 2 Bit Data Size UNSIGNED_BYTE_SIZE — 8 Bit Unsigned Data Size SIGNED_BYTE_SIZE — 8 Bit Signed Data Size UNSIGNED_WORD_SIZE — 16 Bit Unsigned Data Size SIGNED_WORD_SIZE — 16 Bit Signed Data Size UNSIGNED_DWORD_SIZE — 32 Bit Unsigned Data Size SIGNED_DWORD_SIZEE — 32 Bit Signed Data Size FLOAT32_SIZE — 32 Bit Float Data Size
pvData	Pointer to the data of the object.

The ptUpdateParams is a pointer to structure (struct sIEC104UpdateOptionParameters) containing the following fields. If set to NULL only one item is updated at a time.

Field	Description
u16Count	Number of items to be updated.

The time stamp structure (struct sSETTimeStamp) contains the following fields

Field	Description
u8Day	Day of the month. The range is 1 to 31
u8Month	Month of the year. The range is 1 to 12
u16Year	Year. The range is 1970 to 9999
u8DayoftheWeek	Day of the week. 0 is not used. 1=Monday to 7 = Sunday
u8Hour	Hour in the range 0 to 23
u8Minute	Minutes in the range 0 to 59
u8Seconds	Seconds in the range 0 to 59
u16MilliSeconds	Milliseconds in the range 0 to 999
u16MicroSeconds	Microseconds in the range 0 to 999
i8DSTTime	Day light savings time -1 is DST Unknown. The range is 0 to 24

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Update the data attribute ID to the new value specified by the data attribute data. If the value or quality changes than IEC104 APDU is sent. This function must be used only by the server application.

Usage

```
// Sample Update Function
        enum eTgtErrorCodes             eErrorCode      = EC_NONE;
        tErrorValue                     tErrorValue     = EV_NONE;

        struct sIEC104DataAttributeID *psDAID               = NULL; //update dataaddtribute
        struct sIEC104DataAttributeData *psNewValue         = NULL; //updtae new value
        unsigned int uiCount;

        Unsigned8   u8Data                  = 1;
```

```
        Float32 f32Data                 = -10;
        Boolean bGenEvent  = TRUE;
        // update parameters
        uiCount     =   2;
        psDAID      =   calloc(uiCount,sizeof(struct sIEC104DataAttributeID));
        psNewValue  =   calloc(uiCount,sizeof(struct sIEC104DataAttributeData));

        psDAID[0].eTypeID                   =   M_SP_NA_1;
        psDAID[0].u32IOA                    =   5006;
        psDAID[0].pvUserData                =   NULL;
        psNewValue[0].tQuality              =   GD;
        //current date 11/8/2012
        psNewValue[0].sTimeStamp.u8Day          =   8;
        psNewValue[0].sTimeStamp.u8Month        =   11;
        psNewValue[0].sTimeStamp.u16Year        =   2012;
        //time 13.35.0
        psNewValue[0].sTimeStamp.u8Hour         =   13;
        psNewValue[0].sTimeStamp.u8Minute       =   36;
        psNewValue[0].sTimeStamp.u8Seconds      =   0;
        psNewValue[0].sTimeStamp.u16MilliSeconds    =   0;
        psNewValue[0].sTimeStamp.u16MicroSeconds    =   0;
        psNewValue[0].sTimeStamp.i8DSTTime      =   0; //No Day light saving time
        psNewValue[0].sTimeStamp.u8DayoftheWeek     =   4;

        psNewValue[0].pvData                =   &u8Data;
        psNewValue[0].eDataType             =   SINGLE_POINT_DATA;
        psNewValue[0].eDataSize             =   DOUBLE_POINT_SIZE;

        psDAID[1].eTypeID                   =   M_ME_TF_1;
        psDAID[1].u32IOA                    =   7006L;
        psDAID[1].pvUserData                =   NULL;
        psNewValue[1].tQuality              =   GD;
        //current date 11/8/2012
        psNewValue[1].sTimeStamp.u8Day          =   8;
        psNewValue[1].sTimeStamp.u8Month        =   11;
        psNewValue[1].sTimeStamp.u16Year        =   2012;

        //time 13.35.0
        psNewValue[1].sTimeStamp.u8Hour         =   13;
        psNewValue[1].sTimeStamp.u8Minute       =   36;
        psNewValue[1].sTimeStamp.u8Seconds      =   0;
        psNewValue[1].sTimeStamp.u16MilliSeconds    =   0;
        psNewValue[1].sTimeStamp.u16MicroSeconds    =   0;
        psNewValue[1].sTimeStamp.i8DSTTime      =   0; //No Day light saving time
        psNewValue[1].sTimeStamp.u8DayoftheWeek     =   4;

        psNewValue[1].pvData                =   &f32Data;
        psNewValue[1].eDataType             =   FLOAT32_DATA;
        psNewValue[1].eDataSize             =   FLOAT32_SIZE;

        // Update server
        eErrorCode = IEC104Update(myServer,bGenEvent,psDAID,psNewValue,uiCount,&tErrorValue);
//Update myServer
        if(eErrorCode != EC_NONE)
        {
            printf("\r\nError: IEC104Update() failed:  %i %i", eErrorCode, tErrorValue);
        }
```

2.7 IEC104Read

Syntax

enum eAppErrorCodes IEC104Read(IEC104Object myIEC104Obj, struct sIEC104DataAttributeID * psDAID, struct sIEC104DataAttributeData * psReturnedValue, tAppErrorValue * ptErrorValue)

Parameters

Field	Description
myIEC104Obj	IEC104 Object created by IEC104Create
psDAID	Pointer to data attribute ID (See struct sIEC104DataAttributeID)
psReturnedValue	Pointer to data attribute data (See struct sIEC104DataAttributeData)
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Send a read command C_RD_NA_1 for the data attribute ID and return the data attribute data read. This function must be used only by the client application.

Usage

```
// Sample Read Function
        enum eTgtErrorCodes         eErrorCode      = EC_NONE;
        tErrorValue             tErrorValue     = EV_NONE;

        struct sIEC104DataAttributeID sDAID;
        struct sIEC104DataAttributeData sReturnedValue;

            strcpy((char*)sDAID.ai8IPAddress,"127.0.0.1");
                                sDAID.u16PortNumber                     =       2404;
        sDAID.eTypeID           =   M_SP_NA_1;
        sDAID.u16CommonAddress  =   1;
        sDAID.u32IOA            =   8006;

        eErrorCode =    IEC104Read(myClient,&sDAID, &sReturnedValue,&tErrorValue);
        if(eErrorCode != EC_NONE)
        {
            printf("\r\nError: IEC104Read() failed:  %i %i", eErrorCode, tErrorValue);
        }
```

2.8 IEC104Write

Syntax

PUBLICAPIPX enum eTgtErrorCodes PUBLICAPISX IEC104Write(IEC104Object myIEC104Obj, enum eIEC104CounterFreezeFlags eCounterFreeze, struct sIEC104DataAttributeID *psDAID, struct sIEC104DataAttributeData *psWriteValue, struct sIEC104WriteParameters *ptWriteParams, tErrorValue *ptErrorValue)

Parameters

[in]	myIEC104Obj	IEC104 object
[in]	eCounterFreeze	enum eIEC104CounterFreezeFlags
[in]	psDAID	Pointer to IEC104_DataAttributeID structure (or compatible) that identifies the point that is to be written
[in]	psWriteValue	Pointer to Object Data structure that hold the new value of the struct sIEC104DataAttributeData
[in]	ptWriteParams	Pointer to struct sIEC104WriteParameters
[out]	ptErrorValue	Pointer to Error Value

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Sends a clock synchronisation command to the server. This function must be used only by the client application.

Usage

```
// Sample write function
        enum eTgtErrorCodes          eErrorCode      = EC_NONE;
        tErrorValue                  tErrorValue     = EV_NONE;

        struct sIEC104DataAttributeID sDAID;
        struct sIEC104DataAttributeData sWriteValue;

      strcpy((char*)sDAID.ai8IPAddress,"127.0.0.1");
      sDAID.u16PortNumber                    =       2404;
        sDAID.eTypeID          =  C_IC_NA_1;
        sDAID.u16CommonAddress   =  1;
      time(&now);
      timeinfo = localtime(&now);
      timeinfo->tm_year += 1900;

      //current date
      sWriteValue.sTimeStamp.u8Day                  =       (Unsigned8)timeinfo->tm_mday;
      sWriteValue.sTimeStamp.u8Month                =       (Unsigned8)(timeinfo->tm_mon + 1);
      sWriteValue.sTimeStamp.u16Year                =       timeinfo->tm_year;
                              //time
      sWriteValue.sTimeStamp.u8Hour                 =       (Unsigned8)timeinfo->tm_hour;
      sWriteValue.sTimeStamp.u8Minute               =       (Unsigned8)timeinfo->tm_min;
      sWriteValue.sTimeStamp.u8Seconds        =       (Unsigned8)(timeinfo->tm_sec);
```

```
sWriteValue.sTimeStamp.u16MilliSeconds      =      0;
sWriteValue.sTimeStamp.u16MicroSeconds      =      0;
sWriteValue.sTimeStamp.i8DSTTime            =      0; //No Day light saving time
sWriteValue.sTimeStamp.u8DayoftheWeek       =      4;


eErrorCode =    IEC104Write(myClient, IEC104_READ, &sDAID, &sWriteValue,&tErrorValue);
if(eErrorCode != EC_NONE)
{
    printf("\r\nError: IEC104Write() failed:  %i %i", eErrorCode, tErrorValue);
}
```

2.9 IEC104Select

Syntax

```
enum eAppErrorCodes  IEC104Select(IEC104Object myIEC104Obj, struct sIEC104DataAttributeID * psDAID,
                     struct sIEC104DataAttributeData * psSelectValue,
                     struct sIEC104CommandParameters *psSelectParams, tAppErrorValue * ptErrorValue)
```

Parameters

Field	Description
myIEC104Obj	IEC104 Object created by IEC104Create
psDAID	Pointer to data attribute ID (See struct sIEC104DataAttributeID)
psSelectValue	Pointer to data attribute data (See struct sIEC104DataAttributeData)
psSelectParams	Pointer to command parameters (See struct sIEC104CommandParameters)
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

The psSelectParams is a pointer to structure (struct sIEC104CommandParameters) containing the following fields

Field	Description
eQOCQU	Command qualifier must be one of the following NOADDDEF — No additional definition SHORTPULSE — Short pulse duration LONGPULSE — Long pulse duration PERSISTANT — Persistent
u32PulseDuration	Pulse duration in milliseconds based on command qualifier

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Sends a control select command to the server. This function must be used only by the client application.

Usage

```
enum eTgtErrorCodes          eErrorCode      = EC_NONE;
tErrorValue                  tErrorValue     = EV_NONE;

Float32        f32value      =   0;
struct sIEC104DataAttributeID sDAID;
struct sIEC104DataAttributeData sSelectValue;
struct sIEC104CommandParameters sSelectParams;
```

```
strcpy((char*)sDAID.ai8IPAddress,"127.0.0.1");
sDAID.u16PortNumber                                    =          2404;
sDAID.eTypeID              =   C_SE_TC_1;
sDAID.u16CommonAddress    =   1;
sDAID.u32IOA               =   8006;

f32value                   =   -1.2345;
sSelectValue.eDataSize      =   FLOAT32_SIZE;
sSelectValue.eDataType      =   FLOAT32_DATA;
sSelectValue.pvData         =   &f32value;

sSelectParams.eQOCQU        =   NOADDDEF;
memset(&sSelectValue.sTimeStamp, 0, sizeof(struct sSETTimeStamp));

//current date 11/8/2012
sSelectValue.sTimeStamp.u8Day              =   8;
sSelectValue.sTimeStamp.u8Month            =   11;
sSelectValue.sTimeStamp.u16Year            =   2012;

//time 13.35.0
sSelectValue.sTimeStamp.u8Hour             =   13;
sSelectValue.sTimeStamp.u8Minute           =   36;
sSelectValue.sTimeStamp.u8Seconds          =   0;
sSelectValue.sTimeStamp.u16MilliSeconds     =   0;
sSelectValue.sTimeStamp.u16MicroSeconds     =   0;
sSelectValue.sTimeStamp.i8DSTTime          =   0; //No Day light saving time
sSelectValue.sTimeStamp.u8DayoftheWeek      =   4;


eErrorCode =    IEC104Select(myClient,&sDAID, &sSelectValue, &sSelectParams,&tErrorValue);
if(eErrorCode != EC_NONE)
{
    printf("\r\nError: IEC104Select() failed:  %i %i", eErrorCode, tErrorValue);
}
```

2.10 IEC104Operate

Syntax

```
enum eAppErrorCodes  IEC104Operate(IEC104Object myIEC104Obj, struct sIEC104DataAttributeID * psDAID,
                    struct sIEC104DataAttributeData * psOperateValue,
                    struct sIEC104CommandParameters *psOperateParams, tAppErrorValue * ptErrorValue)
```

Parameters

Field	Description
myIEC104Obj	IEC104 Object created by IEC104Create
psDAID	Pointer to data attribute ID (See struct sIEC104DataAttributeID)
psOperateValue	Pointer to data attribute data (See struct sIEC104DataAttributeData)
psOperateParams	Pointer to command parameters (See struct sIEC104CommandParameters)
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Sends a control operate command to the server. This function must be used only by the client application.

Usage

```
// Operate function sample code
        enum eTgtErrorCodes         eErrorCode      = EC_NONE;
        tErrorValue             tErrorValue     = EV_NONE;

        Float32         f32value        =   0;
        struct sIEC104DataAttributeID sDAID;
        struct sIEC104DataAttributeData sOperateValue;
        struct sIEC104CommandParameters sOperateParams;



        strcpy((char*)sDAID.ai8IPAddress,"127.0.0.1");
        sDAID.u16PortNumber                     =       2404;
        sDAID.eTypeID           =   C_SE_TC_1;
        sDAID.u16CommonAddress  =   1;
        sDAID.u32IOA            =   8006;

        f32value                =   -1.2345;
        sOperateValue.eDataSize     =   FLOAT32_SIZE;
        sOperateValue.eDataType     =   FLOAT32_DATA;
        sOperateValue.pvData        =   &f32value;

        sOperateParams.eQOCQU       =   NOADDDEF;
        memset(&sOperateValue.sTimeStamp, 0, sizeof(struct sSETTimeStamp));

        //current date 11/8/2012
        sOperateValue.sTimeStamp.u8Day          =   8;
        sOperateValue.sTimeStamp.u8Month            =   11;
        sOperateValue.sTimeStamp.u16Year            =   2012;

        //time 13.35.0
```

```
        sOperateValue.sTimeStamp.u8Hour            =   13;
        sOperateValue.sTimeStamp.u8Minute          =   36;
        sOperateValue.sTimeStamp.u8Seconds         =   0;
        sOperateValue.sTimeStamp.u16MilliSeconds       =   0;
        sOperateValue.sTimeStamp.u16MicroSeconds       =   0;
        sOperateValue.sTimeStamp.i8DSTTime         =   0; //No Day light saving time
        sOperateValue.sTimeStamp.u8DayoftheWeek    =   4;


    eErrorCode =    IEC104Operate(myClient,&sDAID, &sOperateValue, &sOperateParams,&tErrorValue);
        if(eErrorCode != EC_NONE)
        {
            printf("\r\nError: IEC104Operate() failed:  %i %i", eErrorCode, tErrorValue);
        }
```

2.11 IEC104Cancel

Syntax

PUBLICAPIPX enum eTgtErrorCodes PUBLICAPISX IEC104Cancel(enum eOperationFlag eOperation, IEC104Object myIEC104Obj, struct sIEC104DataAttributeID *psDAID, struct sIEC104DataAttributeData *psCancelValue, struct sIEC104CommandParameters *psCancelParams, tErrorValue *ptErrorValue);

Parameters

Field	Description
eOperation	enum eOperationFlag
myIEC104Obj	IEC104 Object created by IEC104Create
psDAID	Pointer to data attribute ID (See struct sIEC104DataAttributeID)
psCancelValue	Pointer to data attribute data (See struct sIEC104DataAttributeData)
psCancelParams	Pointer to command parameters (See struct sIEC104CommandParameters)
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Sends a control cancel command to the server. This function must be used by client application

Usage

```
// Cancel function sample code
        enum eTgtErrorCodes          eErrorCode      = EC_NONE;
        tErrorValue                  tErrorValue     = EV_NONE;

        Float32        f32value      =  0;
        enum eOperationFlag eOperation = OPERATE;
        struct sIEC104DataAttributeID sDAID;
        struct sIEC104DataAttributeData sCancelValue;
        struct sIEC104CommandParameters sCancelParams;

        strcpy((char*)sDAID.ai8IPAddress,"127.0.0.1");
       sDAID.u16PortNumber                          =        2404;
        sDAID.eTypeID          =  C_SE_TC_1;
        sDAID.u16CommonAddress   =  1;
        sDAID.u32IOA           =  8006;

        f32value               =  -1.2345;
        sCancelValue.eDataSize     =  FLOAT32_SIZE;
        sCancelValue.eDataType     =  FLOAT32_DATA;
        sCancelValue.pvData        =  &f32value;

        sCancelParams.eQOCQU       =  NOADDDEF;
        memset(&sCancelValue.sTimeStamp, 0, sizeof(struct sSETTimeStamp));

        //current date 11/8/2012
        sCancelValue.sTimeStamp.u8Day          =  8;
        sCancelValue.sTimeStamp.u8Month        =  11;
        sCancelValue.sTimeStamp.u16Year        =  2012;

        //time 13.35.0
        sCancelValue.sTimeStamp.u8Hour         =  13;
```

```
        sCancelValue.sTimeStamp.u8Minute          =   36;
        sCancelValue.sTimeStamp.u8Seconds         =   0;
        sCancelValue.sTimeStamp.u16MilliSeconds   =   0;
        sCancelValue.sTimeStamp.u16MicroSeconds   =   0;
        sCancelValue.sTimeStamp.i8DSTTime         =   0; //No Day light saving time
        sCancelValue.sTimeStamp.u8DayoftheWeek    =   4;


eErrorCode =    IEC104Cancel(OPERATE, myClient,&sDAID, &sCancelValue, &sCancelParams,&tErrorValue);
        if(eErrorCode != EC_NONE)
        {
            printf("\r\nError: IEC104Cancel() failed:  %i %i", eErrorCode, tErrorValue);
        }
```

2.12 IEC104GetDataTypeAndSize

Syntax

```
enum eAppErrorCodes  IEC104GetDataTypeAndSize (IEC104Object myIEC104Obj,
                                               struct sIEC104DataAttributeID * psDAID,
                                               struct sIEC104DataAttributeData * psReturnedValue,
                                               tAppErrorValue * ptErrorValue)
```

Parameters

Field	Description
myIEC104Obj	IEC104 Object created by IEC104Create
psDAID	Pointer to data attribute ID (See struct sIEC104DataAttributeID)
psReturnedValue	Pointer to data attribute data (See struct sIEC104DataAttributeData)
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Get the data type and data size for the type ID

Usage

```
// Get data type and size function sample code
enum eAppErrorCodes                eErrorCode          = APP_ERROR_NONE;
tAppErrorValue                     tErrorValue         = APP_ERRORVALUE_NONE;
struct sIEC104DataAttributeID      sDAID               = {0};
struct sIEC104DataAttributeData    sReturnedValue      = {0};

// Set the Type ID for which you want to get the data type and size
sDAID.eTypeID   = C_SC_NA_1;

// Call function to get type and size
eErrorCode = IEC104GetDataTypeAndSize(myIEC104Obj, &sDAID, &sReturnedValue, &tErrorValue);
if(eErrorCode != APP_ERROR_NONE)
{
        printf("Get Type IEC 104 has failed: %i %i", eErrorCode, tErrorValue);
}
else
{
        printf("\r\n Type is : %u, Size is %u", sReturnedValue.eDataType, sReturnedValue.eDataSize);
}
```

2.13 IEC104SetDebugOptions

Syntax

```
enum eAppErrorCodes  IEC104SetDebugOptions(IEC104Object myIEC104Obj,
                                           struct sIEC104DebugParameters *psDebugParams,
                                           tAppErrorValue *ptErrorValue)
```

Parameters

Field	Description
myIEC104Obj	IEC104 Object created by IEC104Create
psDebugParams	Pointer to debug parameters (See struct sIEC104DebugParameters)
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

The debug option could be either function or by the API IEC104LoadConfiguration. If the debug options are set than provided debug call-back will be called.

Usage

```
// Set debug option sample code
enum eAppErrorCodes          eErrorCode          = APP_ERROR_NONE;
tAppErrorValue               tErrorValue         = APP_ERRORVALUE_NONE;
struct sIEC104DebugParameters sDebugParams       = {0};

// Set the debug option to error, transmission and reception data
sDebugParams.u32DebugOptions = DEBUG_OPTION_ERROR | DEBUG_OPTION_TX |
                                                    DEBUG_OPTION_RX;

//Call function to set debug options
eErrorCode = IEC104SetDebugOptions(myIEC104Obj, &sDebugParams, &tErrorValue);
if(eErrorCode != APP_ERROR_NONE)
{
        printf("Set debug options IEC 104 has failed: %i %i", eErrorCode, tErrorValue);
}
```

2.14 IEC104ErrorCodeString

Syntax

PUBLICAPIPX void PUBLICAPISX IEC104ErrorCodeString(struct sIEC104ErrorCode *psIEC104ErrorCodeDes)

Parameters

Field	Description
psIEC104ErrorCodeDes	Pointer to struct sIEC104ErrorCode

Return

APP_ERROR_NONE on success, otherwise

Description

Returns an Error String for the error code provided

Usage

```
// Get error string sample code
        struct sIEC104ErrorCode sIEC104ErrorCodeDes  = {0};
        const char *i8ReturnedMessage = " ";

        sIEC104ErrorCodeDes.iErrorCode = errorcode;

        IEC104ErrorCodeString(&sIEC104ErrorCodeDes);

        i8ReturnedMessage = sIEC104ErrorCodeDes.LongDes;

        return (i8ReturnedMessage);
```

2.15 IEC104ErrorValueString

Syntax

PUBLICAPIPX void PUBLICAPISX IEC104ErrorValueString(struct sIEC104ErrorValue *psIEC104ErrorValueDes);

Parameters

Field	Description
psIEC104ErrorValueDes	Pointer to struct sIEC104ErrorValue

Return

APP_ERROR_NONE on success, otherwise

Description

Returns an Error String for the error value provided

Usage

```
// Get error string sample code
  struct sIEC104ErrorValue sIEC104ErrorValueDes  = {0};
   const char *i8ReturnedMessage = " ";

   sIEC104ErrorValueDes.iErrorValue = errorvalue;

   IEC104ErrorValueString(&sIEC104ErrorValueDes);

   i8ReturnedMessage = sIEC104ErrorValueDes.LongDes;
```

2.16 IEC104ReadCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104ReadCallback) (Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptReadID, struct sIEC104DataAttributeData *ptReadValue, struct sIEC104ReadParameters *ptReadParams, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptReadID**	Pointer to IEC 104 Data Attribute ID
[out]	**ptReadValue**	Pointer to IEC 104 Data Attribute Data
[in]	**ptReadParams**	Pointer to Read parameters
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

This call-back is called when read command (C_RD_NA_1) is received by the server. This function must be used by server application only and is used when client wants to read the current data.

Usage

```
/* Sample read call-back */

enum eAppErrorCodes cbRead(Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptReadID,struct
sIEC104DataAttributeData *ptReadValue, struct sIEC104ReadParameters *ptReadParams, tErrorValue
*ptErrorValue )
   {
      enum eAppErrorCodes     eErrorCode      = APP_ERROR_NONE;
      Unsigned16              u16AnalogVal    = 0;

      // If the type ID and IOA matches increment the analog value and send it.
      if((ptReadID->eTypeID == M_ME_TD_1) && (ptReadID->u32IOA == 5000))
      {
        u16AnalogVal++;
        ptReadParams->pvData = &u16AnalogVal;
      }

     return eErrorCode;
   }
```

2.17 IEC104WriteCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104WriteCallback) (Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptWriteID, struct sIEC104DataAttributeData *ptWriteValue, struct sIEC104WriteParameters *ptWriteParams, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptWriteID**	Pointer to IEC 104 Data Attribute ID
[in]	**ptWriteValue**	Pointer to IEC 104 Data Attribute Data
[in]	**ptWriteParams**	Pointer to Write parameters
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

This call-back is called when clock synchronisation command (C_CS_NA_1) is received by the server. This function must be used by server application only and is used when client wants to set time of the server.

Usage

```
/* Sample write call-back */

 enum eAppErrorCodes cbWrite(Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptWriteID,struct
sIEC104DataAttributeData *ptWriteValue, struct sIEC104WriteParameters *ptWriteParams, tErrorValue
*ptErrorValue )
    {
        enum eAppErrorCodes     eErrorCode      = APP_ERROR_NONE;
        struct sSETTimeStamp    sReceivedTime   = {0};

        // If the type ID is Clock Synchronisation than set time and date based on target
        if(ptWriteID->eTypeID == C_CS_NA_1)
        {
           memcpy(&sReceivedTime, ptWriteValue->sTimeStamp, sizeof(struct sSETTimeStamp));
           SetTimeDate(&sReceivedTime);
       }

       return eErrorCode;
    }
```

2.18 IEC104UpdateCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104UpdateCallback) (Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptUpdateID, struct sIEC104DataAttributeData *ptUpdateValue, struct sIEC104UpdateParameters *ptUpdateParams, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptUpdateID**	Pointer to IEC 104 Data Attribute ID
[in]	**ptUpdateValue**	Pointer to IEC 104 Data Attribute Data
[in]	**ptUpdateParams**	Pointer to Update parameter
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

The ptUpdateParams is a pointer to structure (struct sIEC104UpdateParameters) containing the following fields

Field	Description
eCause	Cause of transmission PERCYC Periodic or Cyclic SPONT Spontaneous INROGEN Interrogated by station interrogation INRO1 Interrogated by group 1 interrogation INRO2 Interrogated by group 2 interrogation INRO3 Interrogated by group 3 interrogation INRO4 Interrogated by group 4 interrogation INRO5 Interrogated by group 5 interrogation INRO6 Interrogated by group 6 interrogation INRO7 Interrogated by group 7 interrogation INRO8 Interrogated by group 8 interrogation INRO9 Interrogated by group 9 interrogation INRO10 Interrogated by group 10 interrogation INRO11 Interrogated by group 11 interrogation INRO12 Interrogated by group 12 interrogation INRO13 Interrogated by group 13 interrogation INRO14 Interrogated by group 14 interrogation INRO15 Interrogated by group 15 interrogation INRO16 Interrogated by group 16 interrogation REQCOGEN Requested by general counter request REQCO1 Requested by group 1 counter request REQCO2 Requested by group 2 counter request REQCO3 Requested by group 3 counter request REQCO4 Requested by group 4 counter request

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

This call-back is called when the APDU's are received by the client. This function must be used by client application only to get the monitor information data from the server.

Usage

```
/* Sample update call-back */

enum eAppErrorCodes cbUpdate(Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptUpdateID, struct
sIEC104DataAttributeData *ptUpdateValue, struct sIEC104UpdateParameters *ptUpdateParams, tErrorValue
*ptErrorValue )
    {
        enum eAppErrorCodes     eErrorCode          = APP_ERROR_NONE;
        Float32                 f32TemperatueVal    = 0.0;

        // Check if we received the type ID and IOA than display the temperature
        if((ptUpdateID->eTypeID == M_ME_TF_1) && (ptUpdateID->u32IOA == 7000))
        {
           memcpy(&f32TemperatueVal, ptUpdateValue->pvData, ptUpdateValue-> eDataSize);
           DisplayTemperature(f32TemperatueVal);
       }

       return eErrorCode;
   }
```

2.19 IEC104ControlSelectCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104ControlSelectCallback) (Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptSelectID, struct sIEC104DataAttributeData *ptSelectValue, struct sIEC104CommandParameters *ptSelectParams, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptSelectID**	Pointer to IEC 104 Data Attribute ID
[in]	**ptSelectValue**	Pointer to IEC 104 Data Attribute Data
[in]	**ptSelectParams**	Pointer to select parameters
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

This call-back is called in the server application when select control command is received from the client.
This function must be used by server application only

Usage

```
/* Sample control select call-back */

 enum eAppErrorCodes cbSelect(Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptSelectID, struct
sIEC104DataAttributeData *ptSelectValue, struct sIEC104CommandParameters *ptSelectParams, tErrorValue
*ptErrorValue )
    {
        enum eAppErrorCodes     eErrorCode          = APP_ERROR_NONE;
        Unsigned8               u8CommandVal        = 0;

        // Check if we received the type ID and IOA and perform Select in the hardware
        if((ptSelectID->eTypeID == C_SC_NA_1) && (ptSelectID->u32IOA == 9000))
       {
           memcpy(&u8CommandVal, ptSelectValue->pvData, ptSelectValue->eDataSize);
           HardwareControlSelect(u8CommandVal);
       }

       return eErrorCode;
   }
```

2.20 IEC104ControlOperateCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104ControlOperateCallback) (Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptOperateID, struct sIEC104DataAttributeData *ptOperateValue, struct sIEC104CommandParameters *ptOperateParams, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptOperateID**	Pointer to IEC 104 Data Attribute ID
[in]	**ptOperateValue**	Pointer to IEC 104 Data Attribute Data
[in]	**ptOperateParams**	Pointer to operate parameters
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise **Error Codes**

Description

This call-back is called in the server application when operate control command is received from the client. This function must be used by server application only

Usage

```
/* Sample control operate call-back */

    enum eAppErrorCodes cbOperate(Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptOperateID,
struct sIEC104DataAttributeData *ptOperateValue, struct sIEC104CommandParameters *ptOperateParams,
tErrorValue *ptErrorValue )
    {
        enum eAppErrorCodes     eErrorCode          = APP_ERROR_NONE;
        Unsigned8               u8CommandVal        = 0;

        // Check if we received the type ID and IOA and perform Operate in the hardware
        if((ptOperateID->eTypeID == C_SC_NA_1) && (ptOperateID->u32IOA == 9000))
        {
            memcpy(&u8CommandVal, ptOperateValue->pvData, ptOperateValue->eDataSize);
            HardwareControlOperate(u8CommandVal);
        }

        return eErrorCode;
    }
```

2.21 IEC104ControlPulseEndActTermCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104ControlPulseEndActTermCallback) (Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptOperateID, struct sIEC104DataAttributeData *ptOperateValue, struct sIEC104CommandParameters *ptOperateParams, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptOperateID**	Pointer to IEC 104 Data Attribute ID
[in]	**ptOperateValue**	Pointer to IEC 104 Data Attribute Data
[in]	**ptOperateParams**	Pointer to pulse end parameters
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise **Error Codes**

Description

This call-back is called in the server application when operate pulse end control command is received from the client.
This function must be used by server application only

Usage

```
/* Sample control operate call-back */

   enum eAppErrorCodes cbPulseEndActTermCallback(Unsigned16 u16ObjectId, struct
sIEC104DataAttributeID *ptOperateID, struct sIEC104DataAttributeData *ptOperateValue, struct
sIEC104CommandParameters *ptOperateParams, tErrorValue *ptErrorValue )
   {
      enum eAppErrorCodes    eErrorCode        = APP_ERROR_NONE;
      Unsigned8              u8CommandVal      = 0;

      // Check if we received the type ID and IOA and perform pulse end Operate in the hardware
      if((ptOperateID->eTypeID == C_SC_NA_1) && (ptOperateID->u32IOA == 9000))
      {
         memcpy(&u8CommandVal, ptOperateValue->pvData, ptOperateValue->eDataSize);
         HardwareControlOperate(u8CommandVal);
      }

      return eErrorCode;
   }
```

2.22 IEC104ControlCancelCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104ControlCancelCallback) (Unsigned16 u16ObjectId, enum eOperationFlag eOperation, struct sIEC104DataAttributeID *ptCancelID, struct sIEC104DataAttributeData *ptCancelValue, struct sIEC104CommandParameters *ptCancelParams, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptCancelID**	Pointer to IEC 104 Data Attribute ID
[in]	**ptCancelValue**	Pointer to IEC 104 Data Attribute Data
[in]	**ptCancelParams**	Pointer to cancel parameters
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise **Error Codes**

Description

This call-back is called in the server application when cancel control command is received from the client. This function must be used by server application only

Usage

```
/* Sample control cancel call-back */

    enum eAppErrorCodes cbCancel(Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptCancelID,
struct sIEC104DataAttributeData *ptCancelValue, struct sIEC104CommandParameters *ptCancelParams,
tErrorValue *ptErrorValue )
    {
        enum eAppErrorCodes     eErrorCode          = APP_ERROR_NONE;
        Unsigned8               u8CommandVal        = 0;

        // Check if we received the type ID and IOA and perform Cancel in the hardware
        if((ptCancelID->eTypeID == C_SC_NA_1) && (ptCancelID->u32IOA == 9000))
       {
           memcpy(&u8CommandVal, ptCancelValue->pvData, ptCancelValue->eDataSize);
           HardwareControlCancel(u8CommandVal);
       }

       return eErrorCode;
    }
```

2.23 IEC104DebugMessageCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104DebugMessageCallback) (Unsigned16 u16ObjectId, struct sIEC104DebugData *ptDebugData, tErrorValue *ptErrorValue)

Parameters

[in] **u16ObjectId** IEC104 object identifier

[in] **ptDebugData** Pointer to debug data

[out] **ptErrorValue** Pointer to Error Value (if any error occurs while creating the object)

The debug data structure (struct sIEC104DebugData) has following fields

Field	Description
u32DebugOptions	Debug option (See u32DebugOptions)
eErrorCode	Returned error code
tErrorValue	Returned error value
u16RxCount	Receive data count
u16TxCount	Transmitted data count
au8ErrorMessage	Error Message string (max 255 characters)
au8WarningMessage	Warning Message string (max 255 characters)
au8RxData	Received Data (max 255 characters)
au8TxData	Transmitted Data (max 255 characters)

Return

APP_ERROR_NONE on success, otherwise **Error Codes**

Description

This call-back is called in the server or client application based on debug options.

Usage

```
/* Sample debug call-back */

    enum eAppErrorCodes cbDebug(Unsigned16 u16ObjectId, struct sIEC104DebugData *ptDebugData,
tErrorValue *ptErrorValue )
    {
        enum eAppErrorCodes     eErrorCode          = APP_ERROR_NONE;
        Unsigned16  nu16Count = 0;
        // If Debug Option set is Rx DATA than print receive data
        if((ptDebugData->u32DebugOptions & DEBUG_OPTION_RX) == DEBUG_OPTION_RX)
        {
           printf("\r\n Rx :");
           for(nu16Count = 0;  nu16Count < ptDebugData->u16RxCount; u16RxCount++)
           {
               printf(" %02X", ptDebugData->au8RxData[nu16Count];
           }
       }

       // If Debug Option set is Tx DATA than print transmission data
```

```
    if((ptDebugData->u32DebugOptions & DEBUG_OPTION_TX) == DEBUG_OPTION_TX)
    {
        printf("\r\n Tx :");
        for(nu16Count = 0;  nu16Count < ptDebugData->u16TxCount; u16TxCount++)
        {
            printf(" %02X", ptDebugData->au8TxData[nu16Count];
        }
    }

    return eErrorCode;
}
```

2.24 IEC104ControlFreezeCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104ControlFreezeCallback) (Unsigned16 u16ObjectId, enum eIEC104CounterFreezeFlags eCounterFreeze, struct sIEC104DataAttributeID *ptFreezeID, struct sIEC104DataAttributeData *ptFreezeValue, struct sIEC104WriteParameters *ptFreezeCmdParams, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptFreezeID**	Pointer to IEC 104 Data Attribute ID
[in]	**ptFreezeValue**	Pointer to IEC 104 Data Attribute Data
[in]	**ptFreezeParams**	Pointer to Freeze parameters
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise **Error Codes**

Description

This call-back is called in the server application when freeze control command is received from the client.
This function must be used by server application only

Usage

```
/* Sample control Freeze call-back */

enum eAppErrorCodes cbControlFreezeCallback(Unsigned16 u16ObjectId, enum eIEC104CounterFreezeFlags
eCounterFreeze, struct sIEC104DataAttributeID *ptFreezeID,  struct sIEC104DataAttributeData *ptFreezeValue,
struct sIEC104WriteParameters *ptFreezeCmdParams, tErrorValue *ptErrorValue )
    {
        enum eAppErrorCodes     eErrorCode      = APP_ERROR_NONE;
        struct sSETTimeStamp    sReceivedTime   = {0};

        // get freeze counter interrogation groub & process it in harware level
```

```
    return eErrorCode;
}
```

2.25 IEC104ParameterActCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104ParameterActCallback) (Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptOperateID, struct sIEC104DataAttributeData *ptOperateValue, struct sIEC104ParameterActParameters *ptParameterActParams, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptOperateID**	Pointer to IEC 104 Data Attribute ID
[in]	**ptOperateValue**	Pointer to IEC 104 Data Attribute Data
[in]	**ptParameterActParams**	Pointer to Parameter Act Params
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise **Error Codes**

Description

This call-back is called in the server application when parameter activation control command is received from the client.
This function must be used by server application only

Usage

```
//Sample Parameter Act call-back

enum eAppErrorCodes cbParameterAct(Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptOperateID,
struct sIEC104DataAttributeData *ptOperateValue,struct sIEC104ParameterActParameters
*ptParameterActParams, tErrorValue *ptErrorValue)
{
    enum eAppErrorCodes    eErrorCode        = APP_ERROR_NONE;
    Unsigned8              u8CommandVal      = 0;
    // print & process parameter value for particular typeid & ioa

    return eErrorCode;
}
```

2.26 IEC104DirectoryCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104DirectoryCallback) (Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptDirectoryID, struct sIEC104DirectoryList *ptDirList, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[in]	**ptDirectoryID**	Pointer to IEC 104 Data Attribute ID
[in]	**ptDirList**	Pointer to IEC 104 **sIEC104DirectoryList**
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise **Error Codes**

Description

This call-back is called in the client application when directory read control is received from the server.
This function must be used by client application only

Usage

```
enum eTgtErrorCodes cbDirectory(struct sIEC104DataAttributeID * psDirectoryID,
                        const struct sIEC104DirectoryList *psDirList,  tErrorValue * ptErrorValue)
{
   enum eTgtErrorCodes eErrorCode                =  EC_NONE;
         Unsigned16                    u16UptoFileCount =  ZERO;

         printf("\n Directory CallBack Called");

         printf("\r\n server ip %s",psDirectoryID->ai8IPAddress);
         printf("\r\n server port %u",psDirectoryID->u16PortNumber);
         printf("\r\n server ca %u",psDirectoryID->u16CommonAddress);
         printf("\r\n Data Attribute ID is  %u IOA %u ",psDirectoryID->eTypeID, psDirectoryID->u32IOA);

         printf("\n No Of Files in the Directory :%u", psDirList->u16FileCount);

         u16UptoFileCount = 0;
         while(u16UptoFileCount < psDirList->u16FileCount)
         {
printf("\n \n Object Index:%u       File Name:%u    SizeofFile:%lu", u16UptoFileCount, psDirList-
>psFileAttrib[u16UptoFileCount].u16FileName, psDirList->psFileAttrib[u16UptoFileCount].zFileSize);
printf("\n Time:%02d:%02d:%02d:%02d:%02d",  psDirList-
>psFileAttrib[u16UptoFileCount].sLastModifiedTime.u8Hour, psDirList-
>psFileAttrib[u16UptoFileCount].sLastModifiedTime.u8Minute, psDirList-
>psFileAttrib[u16UptoFileCount].sLastModifiedTime.u8Seconds, psDirList-
>psFileAttrib[u16UptoFileCount].sLastModifiedTime.u16MilliSeconds, psDirList-
>psFileAttrib[u16UptoFileCount].sLastModifiedTime.u16MicroSeconds);
```

```
                    printf(" Date:%02d:%02d:%04d:%02d\n", psDirList-
>psFileAttrib[u16UptoFileCount].sLastModifiedTime.u8Day, psDirList-
>psFileAttrib[u16UptoFileCount].sLastModifiedTime.u8Month,psDirList-
>psFileAttrib[u16UptoFileCount].sLastModifiedTime.u16Year,psDirList-
>psFileAttrib[u16UptoFileCount].sLastModifiedTime.u8DayoftheWeek);
                    u16UptoFileCount++;
          }

          return eErrorCode;
}
```

2.27 IEC104ClientStatusCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104ClientStatusCallback) (Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptDataID, enum eStatus *peSat, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[out]	**ptDataID**	Pointer to IEC 104 Data Attribute ID
[out]	**peSat**	Pointer to enum eStatus
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise **Error Codes**

Description

This call-back is called in the client application , when the connection status change to connect / not connected

Usage

```
   //Sample client status call-back

        enum eTgtErrorCodes cbClientstatus(Unsigned16 u16ObjectId, struct sIEC104DataAttributeID *ptDataID,
enum eStatus *peSat, tErrorValue *ptErrorValue)
        {
            enum eTgtErrorCodes eErrorCode = EC_NONE;

            do
            {
               printf("\r\n server ip address %s ", ptDataID->ai8IPAddress);
              printf("\r\n server port number %u", ptDataID->u16PortNumber);
              printf("\r\n server ca %u", ptDataID->u16CommonAddress);

              if(*peSat  ==  NOT_CONNECTED)
              {
```

```
                printf("\r\n not connected");
            }
            else
            {
             printf("\r\n connected");
             }

        }while(FALSE);

          return eErrorCode;
    }
```

2.28 IEC104ServerStatusCallback

Syntax

typedef enum eTgtErrorCodes(* IEC104ServerStatusCallback) (Unsigned16 u16ObjectId, struct sIEC104ServerConnectionID *ptServerConnID, enum eStatus *peSat, tErrorValue *ptErrorValue)

Parameters

[in]	**u16ObjectId**	IEC104 object identifier
[out]	**ptServerConnID**	Pointer to struct **sIEC104ServerConnectionID**
[out]	**peSat**	Pointer to enum eStatus
[out]	**ptErrorValue**	Pointer to Error Value (if any error occurs while creating the object)

Return

APP_ERROR_NONE on success, otherwise **Error Codes**

Description

This call-back is called in the server application , when the connection status change to connect / not connected

Usage

```
   //Sample client status call-back

    enum eTgtErrorCodes cbServerStatus(Unsigned16 u16ObjectId, struct sIEC104ServerConnectionID
*ptServerConnID, enum eStatus *peSat, tErrorValue *ptErrorValue)
            {
                enum eTgtErrorCodes eErrorCode = EC_NONE;


               printf("\r\n cbServerstatus() called");
                if(*peSat == CONNECTED)
                {
                printf("\r\n status - connected");
                }
                else
                {
```

```
                printf("\r\n status - disconnected");
                 }

                 printf("\r\n source ip %s port %u ", ptServerConnID->ai8SourceIPAddress, ptServerConnID-
>u16SourcePortNumber);
                printf("\r\n remote ip %s port %u ", ptServerConnID->ai8RemoteIPAddress, ptServerConnID-
>u16RemotePortNumber);

                return eErrorCode;
            }
```

2.29 IEC104GetFile

Syntax

PUBLICAPIPX enum eTgtErrorCodes PUBLICAPISX IEC104GetFile(IEC104Object myIEC104Obj, struct sIEC104DataAttributeID * psDAID, Unsigned16 u16FileName, tErrorValue *ptErrorValue);

Parameters

[in]	**myIEC104Obj**	IEC104 object to Get file transfer
[in]	**psDAID**	Pointer to IEC104 Data Attribute ID
[in]	**u16FileName**	File Name.
[out]	**ptErrorValue**	Pointer to Error Value

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

IEC104 Get file Using File Name.

Usage

Client Example Usage:
\code

```
enum eTgtErrorCodes                                    eErrorCode          = EC_NONE;
tErrorValue                                            tErrorValue     = EV_NONE;
struct sIEC104DataAttributeID sWriteDAID               =       {0};
Unsigned16    u16FileName                                                  = 0;

strcpy((char*)sWriteDAID.ai8IPAddress,"127.0.0.1");
sWriteDAID.u16PortNumber     =       2404;
sWriteDAID.u16CommonAddress =     0;
sWriteDAID.u32IOA                    =       0;
u16FileName = 1042;

printf("\r\n Going for File Tranfer");
eErrorCode = IEC104GetFile(myClient, &sWriteDAID, u16FileName,  &tErrorValue);
if(eErrorCode != EC_NONE)
{
        printf("\r\n Error File Transfer Failed: %i %i", eErrorCode,  tErrorValue);
        break;
}
else
{
                printf("\r\n File Tranfer Success\n\n");
}
\endcode
```

2.30 IEC104SendFile

Syntax

PUBLICAPIPX Integer16 PUBLICAPISX IEC104SendFile(IEC104Object myIEC104Obj, struct sIEC104DataAttributeID *psDAID, Unsigned16 u16FileName, tErrorValue *ptErrorValue);

Parameters

[in]	**myIEC104Obj**	IEC104 object to file transfer in control direction
[in]	**psDAID**	Pointer to IEC104 Data Attribute ID
[in]	**u16FileName**	File Name.
[out]	**ptErrorValue**	Pointer to Error Value

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

IEC104 Send File- Client Send file to Server.

Usage

Client Example Usage:
\code

```
Integer16          iErrorCode       = EC_NONE;
   tErrorValue                tErrorValue      = EV_NONE;
   struct sIEC104DataAttributeID sWriteDAID     =  {0};
   Unsigned16   u16FileName                     = 0;

   strcpy((char*)sWriteDAID.ai8IPAddress,"127.0.0.1");
   sWriteDAID.u16PortNumber  =   2404;
   sWriteDAID.u16CommonAddress =     0;
   sWriteDAID.u32IOA            =   0;
   u16FileName = 1042;

   printf("\r\n Going for File Tranfer");
   iErrorCode = IEC104SendFile(myClient, &sWriteDAID, u16FileName,  &tErrorValue);
   if(iErrorCode != EC_NONE)
   {
     printf("\r\n Error File Transfer Failed: %i %i", iErrorCode,  tErrorValue);
     break;
   }
   else
   {
       printf("\r\n File Tranfer Success\n\n");
   }
\endcode
```

2.31 IEC104ClientStatus

Syntax

PUBLICAPIPX enum eTgtErrorCodes PUBLICAPISX IEC104ClientStatus(IEC104Object myIEC104Obj, struct sIEC104DataAttributeID *psDAID, enum eStatus *peSat, tErrorValue *ptErrorValue);

Parameters

[in]	**myIEC104Obj**	IEC104 object
[in]	**psDAID**	Pointer to IEC104 Data Attribute ID
[out]	**peSat**	Pointer to enum eStatus
[out]	**ptErrorValue**	Pointer to Error Value

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Get IEC104 Client Connection Status

Usage

Example Usage:

```
\code

enum eTgtErrorCodes                    eErrorCode      = EC_NONE;
tErrorValue                            tErrorValue             = EV_NONE;
enum eStatus eSat = 0;
  struct sIEC104DataAttributeID sDAID          =        {0};

        strcpy((char*)sDAID.ai8IPAddress,"127.0.0.1");
        sDAID.u16PortNumber  =        2404;
        sDAID.u16CommonAddress      =        1;
        sDAID.u32IOA        =   0;



eErrorCode = IEC104ClientStatus(myClient, &sDAID, &eSat, &tErrorValue);
        if(eErrorCode != EC_NONE)
        {
                printf("\r\n IEC104ClientStatus Failed: %i %i", eErrorCode,  tErrorValue);
                break;
        }
        else
        {
                        printf("\r\n IEC104ClientStatus  Success\n\n\n");
        }

\endcode
```

2.32 IEC104ClientChangeState

Syntax

PUBLICAPIPX enum eTgtErrorCodes PUBLICAPISX IEC104ClientChangeState(IEC104Object myIEC104Obj, struct sIEC104DataAttributeID *psDAID, enum eConnectState eState, tErrorValue *ptErrorValue);

Parameters

[in]	**myIEC104Obj**	IEC104 object
[in]	**psDAID**	Pointer to IEC104 Data Attribute ID
[in]	**eState**	enum eConnectState
[out]	**ptErrorValue**	Pointer to Error Value

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

IEC104 Client Change State - data mode/ test mode.

Usage

Example Usage:

```
\code

enum eTgtErrorCodes                          eErrorCode        = EC_NONE;
tErrorValue                                  tErrorValue             = EV_NONE;
enum eConnectState eState = TEST_MODE;
   struct sIEC104DataAttributeID sDAID          =        {0};

        strcpy((char*)sDAID.ai8IPAddress,"127.0.0.1");
        sDAID.u16PortNumber  =        2404;
        sDAID.u16CommonAddress      =        1;
        sDAID.u32IOA        =   0;



         eErrorCode = IEC104ClientChangeState(myClient, &sDAID, eState, &tErrorValue);
         if(eErrorCode != EC_NONE)
         {
                printf("\r\n IEC104ClientStatus Failed: %i %i", eErrorCode,  tErrorValue);
                break;
         }
         else
         {
                        printf("\r\n IEC104ClientChangeState  Success\n\n\n");
         }

\endcode
```

2.33 GetIEC104ObjectStatus

Syntax

PUBLICAPIPX enum eTgtErrorCodes PUBLICAPISX GetIEC104ObjectStatus(IEC104Object myIEC104Obj, enum eAppState *peCurrentState, tErrorValue *ptErrorValue);

Parameters

[in]	**myIEC104Obj**	IEC104 object
[out]	**peCurrentState**	Pointer to enum eAppState
[out]	**ptErrorValue**	Pointer to Error Value

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Get IEC104 object Status.

Usage

Example Usage:

```
\code

enum eTgtErrorCodes                    eErrorCode        = EC_NONE;
tErrorValue                            tErrorValue       = EV_NONE;
enum eAppState eCurrentState = 0,


        eErrorCode = GetIEC104ObjectStatus(myClient, &eCurrentState, &tErrorValue);
        if(eErrorCode != EC_NONE)
        {
                printf("\r\nGetIEC104ObjectStatus Failed: %i %i", eErrorCode, tErrorValue);
                break;
        }
        else
        {
                        printf("\r\n GetIEC104ObjectStatus Success\n\n\n");
        }

\endcode
```

2.34 IEC104ListDirectory

Syntax

PUBLICAPIPX enum eTgtErrorCodes PUBLICAPISX IEC104ListDirectory(IEC104Object myIEC104Obj, struct sIEC104DataAttributeID * psDAID, tErrorValue *ptErrorValue);

Parameters

[in]	**myIEC104Obj**	IEC104 object to Get Type and Size
[in]	**psDAID**	Pointer to IEC104 Data Attribute ID
[out]	**ptErrorValue**	Pointer to Error Value

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

IEC104 List Directory.
Get Directory List as call Backs

Usage

Client Example Usage:
\code

```
enum eTgtErrorCodes                                    eErrorCode          = EC_NONE;
tErrorValue                                            tErrorValue         = EV_NONE;
struct sIEC104DataAttributeID sWriteDAID               =       {0};

        strcpy((char*)sWriteDAID.ai8IPAddress,"127.0.0.1");
        sWriteDAID.u16PortNumber      =       2404;
        sWriteDAID.u16CommonAddress           =       1;
        sWriteDAID.u32IOA          =   0;

        eErrorCode = IEC104ListDirectory(myClient, &sWriteDAID, &tErrorValue);
        if(eErrorCode != EC_NONE)
        {
                printf("\r\n Error List Directory Failed: %i %i", eErrorCode,  tErrorValue);
                break;
        }
        else
        {
                        printf("\r\n List Directory  Success\n\n\n");
        }

        \endcode
```

2.35 IEC104Stop

Syntax

enum eAppErrorCodes IEC104Stop (IEC104Object myIEC104Obj, tAppErrorValue * ptErrorValue)

Parameters

Field	Description
myIEC104Obj	IEC104 Object created by IEC104Create
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Stops the IEC 104 object. This stops all the communication

Usage

```
enum eAppErrorCodes        eErrorCode     = APP_ERROR_NONE;
tAppErrorValue             tErrorValue    = APP_ERRORVALUE_NONE;

/* Stop the IEC104 Object */
eErrorCode = IEC104Stop(myIEC104Obj, &tErrorValue);
if(eErrorCode != APP_ERROR_NONE)
{
        printf("Stop IEC104 has failed: %i %i", eErrorCode, tErrorValue);
 }
```

2.36 IEC104ParameterAct

Syntax

PUBLICAPIPX enum eTgtErrorCodes PUBLICAPISX IEC104ParameterAct(IEC104Object myIEC104Obj, struct sIEC104DataAttributeID *psDAID, struct sIEC104DataAttributeData *psOperateValue, struct sIEC104ParameterActParameters *psParaParams, tErrorValue *ptErrorValue);

Parameters

[in]	**myIEC104Obj**	IEC104 object
[in]	**psDAID**	Pointer to IEC104 Data Attribute ID of control that is to be Operated
[in]	**psOperateValue**	Pointer to IEC104 Data Attribute Data (The value the control is to be set)
[in]	**psParaParams**	Pointer to IEC104 Data Attribute Parameters (Quality Parameters)
[out]	**ptErrorValue**	Pointer to Error Value

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Send an Parameter Act command on given control Data object

Usage

```
enum eTgtErrorCodes                    eErrorCode          = EC_NONE;
tErrorValue                                tErrorValue      = EV_NONE;

Float32                    f32value            =      0;
struct sIEC104DataAttributeID sDAID;
struct sIEC104DataAttributeData sOperateValue;
struct sIEC104ParameterActParameters sParaParams;

sDAID.eTypeID                  =      P_ME_NC_1;
sDAID.u16CommonAddress     =  1;
sDAID.u32IOA                   =      8006;

f32value                       =      -1.2345;
sOperateValue.eDataSize    =      FLOAT32_SIZE;
sOperateValue.eDataType    =      FLOAT32_DATA;
sOperateValue.pvData           =      &f32value;

sOperateParams.eQOCQU          =      NOADDDEF;
memset(&sOperateValue.sTimeStamp, 0, sizeof(struct sSETTimeStamp));

//current date 11/8/2012
sOperateValue.sTimeStamp.u8Day                 =      8;
sOperateValue.sTimeStamp.u8Month               =      11;
sOperateValue.sTimeStamp.u16Year               =      2012;

//time 13.35.0
sOperateValue.sTimeStamp.u8Hour            =      13;
sOperateValue.sTimeStamp.u8Minute          =      36;
```

```
                sOperateValue.sTimeStamp.u8Seconds              =       0;
                sOperateValue.sTimeStamp.u16MilliSeconds        =       0;
                sOperateValue.sTimeStamp.u16MicroSeconds        =       0;
                sOperateValue.sTimeStamp.i8DSTTime              =       0; //No Day light saving time
                sOperateValue.sTimeStamp.u8DayoftheWeek   =     4;


eErrorCode =    IEC104ParameterAct(myClient,&sDAID, &sOperateValue, &sOperateParams,&tErrorValue);
                if(eErrorCode != EC_NONE)
                {
                        printf("\r\nError: IEC104ParameterAct() failed:    %i %i", eErrorCode, tErrorValue);
                }
```

2.37 IEC104Free

Syntax

enum eAppErrorCodes IEC104Free (IEC104Object myIEC104Obj, tAppErrorValue * ptErrorValue)

Parameters

Field	Description
myIEC104Obj	IEC104 Object created by IEC104Create
ptErrorValue	Pointer to error value. APP_ERRORVALUE_NONE on success, otherwise Error Values

Return

APP_ERROR_NONE on success, otherwise Error Codes

Description

Frees the IEC 104 object and all the memory allocated

Usage

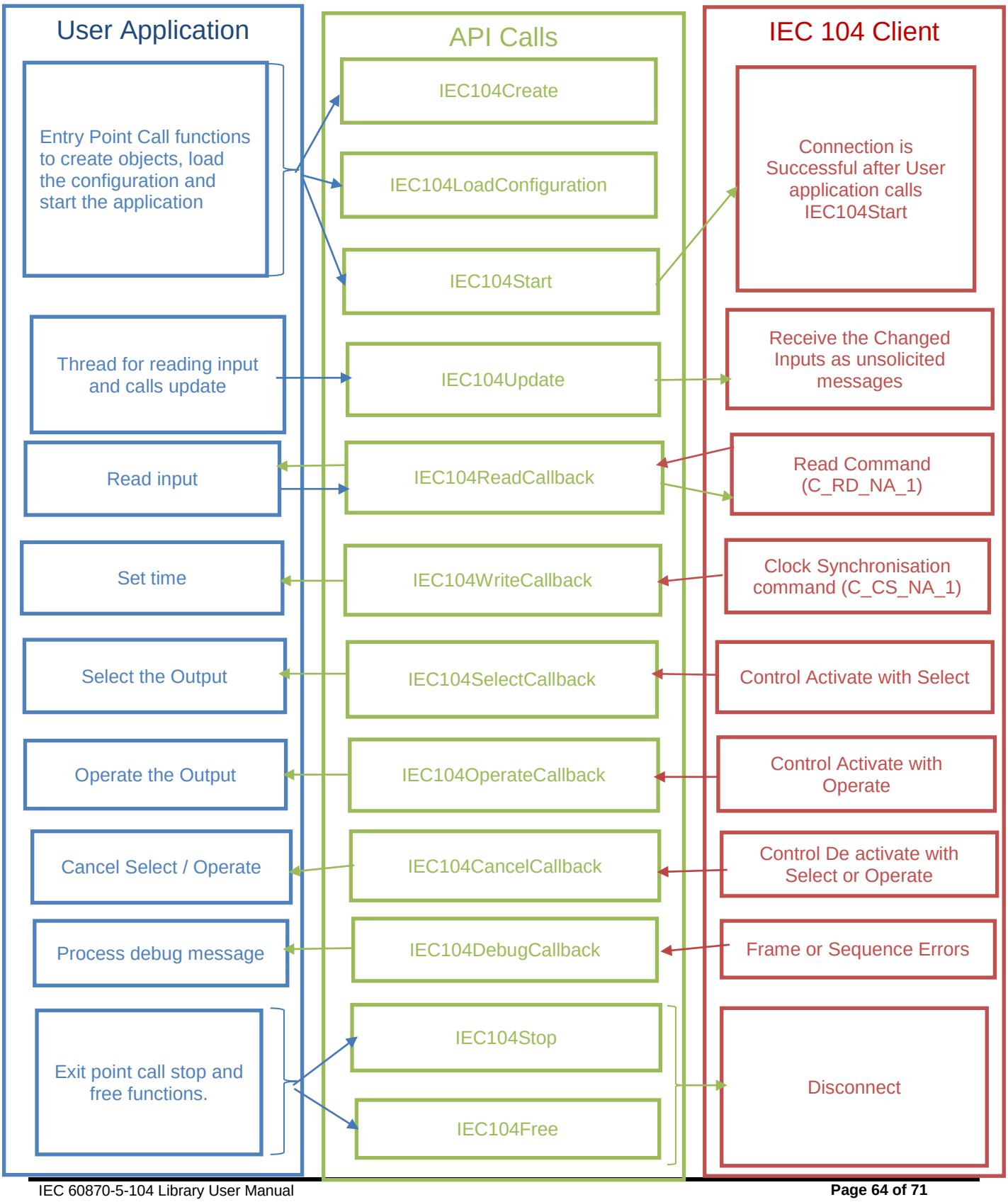
```
enum eAppErrorCodes        eErrorCode     = APP_ERROR_NONE;
tAppErrorValue             tErrorValue    = APP_ERRORVALUE_NONE;

/* Free the IEC104 Object */
eErrorCode = IEC104Free(myIEC104Obj, &tErrorValue);
if(eErrorCode != APP_ERROR_NONE)
{
        printf("Free IEC104 has failed: %i %i", eErrorCode, tErrorValue);
 }
```

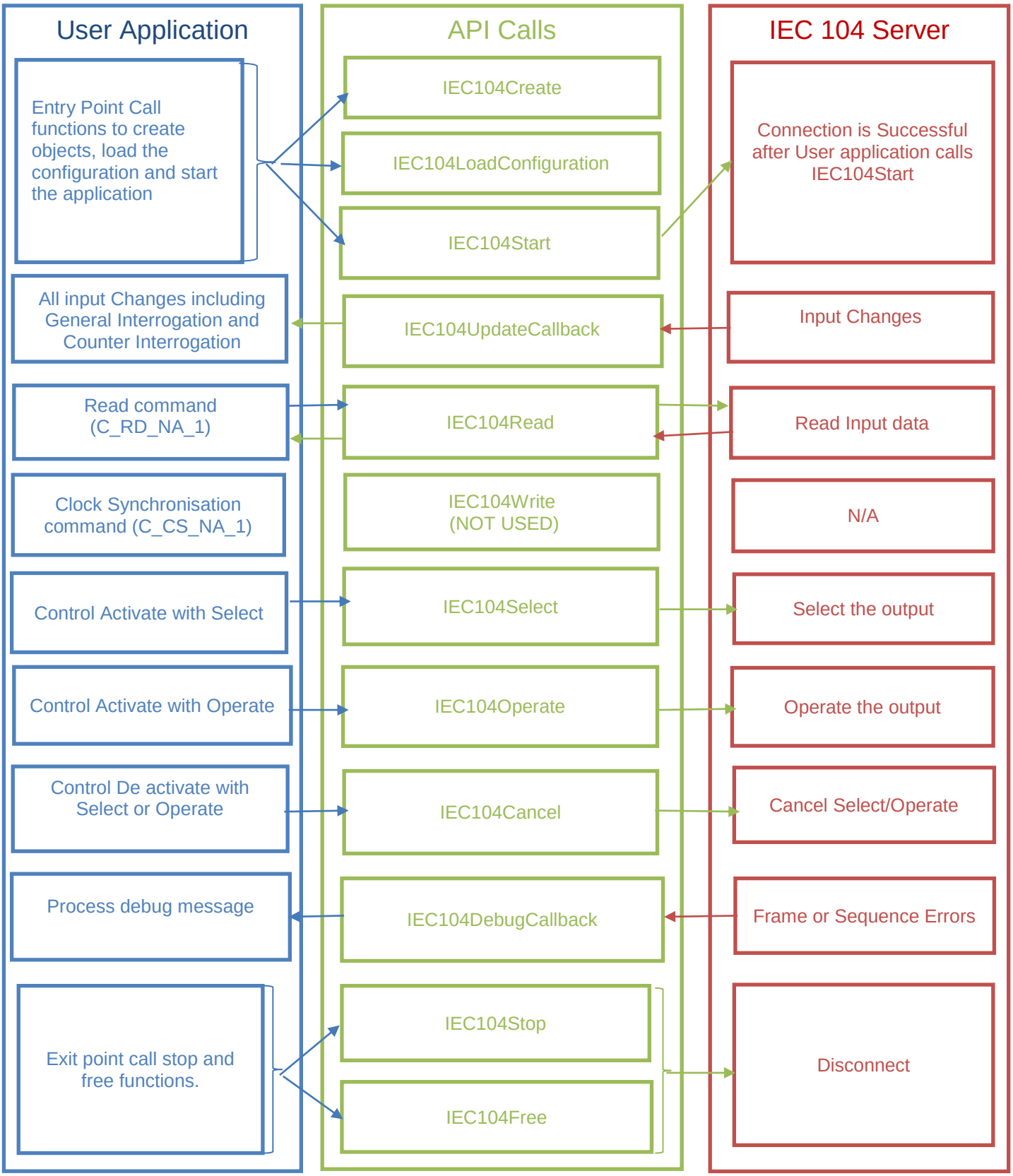
3 Flow Diagrams

The typical flow diagram of user application and the IEC104 Library is described in the sections below

3.1 Server



3.2 Client


User Application
API Calls
IEC 104 Server
Entry Point Call functions to create objects, load the configuration and start the application
IEC104Create
IEC104LoadConfiguration
IEC104Start
Connection is Successful after User application calls IEC104Start
All input Changes including General Interrogation and Counter Interrogation
IEC104UpdateCallback
Input Changes
Read command (C_RD_NA_1)
IEC104Read
Read Input data
Clock Synchronisation command (C_CS_NA_1)
IEC104Write (NOT USED)
N/A
Control Activate with Select
IEC104Select
Select the output
Control Activate with Operate
IEC104Operate
Operate the output
Control De activate with Select or Operate
IEC104Cancel
Cancel Select/Operate
Process debug message
IEC104DebugCallback
Frame or Sequence Errors
Exit point call stop and free functions.
IEC104Stop
IEC104Free
Disconnect

4 Trouble Shooting

This section lists all the error codes and error values

4.1 Error Codes

Error Code	Description	Detail
0	EC_NONE	Everything is ok
-1	EC_PARAMETER	Invalid Parameter
-2	EC_ERROR_VALUE_NULL	Pointer to Error Value is Null
-100	EC_MEMORY	Memory Error
-150	EC_TASK_CREATE	Tasks Create Error
-151	EC_TASK_DELETE	Tasks Delete Error
-152	EC_TASK_ASSIGN_MEM	Error in Task Assign Memory
-200	EC_SEM_CREATE	Semaphore Create Error
-201	EC_SEM_GET	Semaphore Get Error
-202	EC_SEM_RELEASE	Semaphore Release Error
-203	EC_SEM_DESTROY	Semaphore Destroy Error
-204	EC_SEM_FIND	Unable To Find Semaphore
-250	EC_TIMER_CREATE	Timer Create Error
-251	EC_TIMER_START	Timer Start Error
-252	EC_TIMER_STOP	Timer Stop Error
-253	EC_TIMER_RESET	Timer Reset Error
-254	EC_TIMER_DESTROY	Timer Start Error
-300	EC_SOCKET_INITIAL	Socket Initialization Error
-301	EC_SOCKET_CREATE	Socket Start Error
-302	EC_SOCKET_CLOSE	Socket Close Error
-303	EC_SOCKET_SHUTDOWN	Socket Shut Down Error
-304	EC_SOCKET_NONBLOCKING	Socket is non blocking
-305	EC_SOCKET_BIND	Socket Bind Error
-306	EC_SOCKET_LISTEN	Socket Listen Error
-307	EC_SOCKET_ACCEPT	Socket Accept Error
-308	EC_SOCKET_CONNECT	Socket Connect Error
-309	EC_SOCKET_SEND	Socket Send Error
-310	EC_SOCKET_RECEIVE	Socket Receive Error
-311	EC_SOCKET_SEND_TO	Socket SendTo Error
-312	EC_SOCKET_RECEIVE_FROM	Socket Receive From Error
-313	EC_SOCKET_SELECT	Socket Wait for Activity Error
-314	EC_SOCKET_READ_READY	Socket is not ready to read
-315	EC_SOCKET_CONNECT_NOT_SUCEED	Socket Connect is not successful
-316	EC_SOCKET_KEEP_ALIVE	Socket Keep alive failed
-350	EC_SLEEP	Sleep Failed

-351	EC_SET_TIME	Error Set System Time
-352	EC_GET_TIME	Error Get System Time
-400	EC_FILE_OPEN	File Open Error
-401	EC_FILE_CLOSE	File Close Error
-402	EC_FILE_READ	File Read Error
-403	EC_FILE_WRITE	File Write Error
-404	EC_FILE_END	File End Error
-405	EC_FILE_RENAME	Error File Rename
-406	EC_CREATE_DIRECTORY	Error Create Directory
-407	EC_REMOVE_DIRECTORY	Error Remove Directory
-408	EC_CHANGE_DIRECTORY	Error Change Directory
-409	EC_GET_CURRENT_DIRECTORY	Error Get Current Working Directory
-410	EC_FILE_DELETE	Error Delete File
-411	EC_FILE_FIND	Error Find File
-412	EC_FILE_SIZE	Error File Size
-413	EC_FILE_SEEK	Error File Seek
-414	EC_LIST_DIRECTORY	Error List Directory
-450	EC_LIST_ADD	Link List Add Failed
-451	EC_LIST_INSERT	Link List Insert Failed
-500	EC_SERIAL_OPEN	Error in Serial Open
-501	EC_SERIAL_CLOSE	Error in Serial Close
-502	EC_SERIAL_TRANSMIT	Error in Serial Transmit
-503	EC_SERIAL_RECEIVE	Error in Serial Receive
-550	EC_MESSAGE_CREATE	Error in Message Create
-551	EC_MESSAGE_DESTROY	Error in Message Destroy
-552	EC_MESSAGE_SEND	Error in Message Send*
-553	EC_MESSAGE_RECEIVE	Error in Message Receive
-554	EC_MESSAGE_FIND	Error in Message Find
-600	EC_UTILITY_FILEPOSTION	File Position Error
-601	EC_UTILITY_GETSTRING	File Get String Error
-602	EC_UTILITY_SECTIONNAME	File Section Name Error
-603	EC_UTILITY_ITEMNAME	File Item and Text Error
-604	EC_UTILITY_CREATECOMMONMEMORY	Create common memory failed
-605	EC_UTILITY_INITCOMMONMEMORY	Initialize common memory failed
-606	EC_UTILITY_DESTROYCOMMONMEMORY	Destroy common memory failed
-607	EC_UTILITY_GET_MAC_ADDRESS	Get MAC Address FAILED
-608	EC_UTILITY_SYSTEM_REBOOT	System Reboot Failed
-650	EC_GPIO_OPEN	Error in GPIO Open
-651	EC_GPIO_READ	Error in GPIO Read
-652	EC_GPIO_WRITE	Error in GPIO Write
-4501	APP_ERROR_ERRORVALUE_IS_NULL	IEC104 Error value is null
-4502	APP_ERROR_CREATE_FAILED	IEC104 api create function failed
-4503	APP_ERROR_FREE_FAILED	IEC104 api free function failed
-4504	APP_ERROR_INITIALIZE	IEC104 server/client initialize function failed

-4505	APP_ERROR_LOADCONFIG_FAILED	IEC104 apiLoad configuration function failed
-4506	APP_ERROR_CHECKALLOCLOGICNODE	IEC104 Load- check alloc logical node function failed
-4507	APP_ERROR_START_FAILED	IEC104 api Start function failed
-4508	APP_ERROR_STOP_FAILED	IEC104 api Stop function failed
-4509	APP_ERROR_SETDEBUGOPTIONS_FAILED	IEC104 set debug option failed
-4510	APP_ERROR_PHYSICALINITIALIZE_FAILED	IEC104 Physical Layer initialization failed
-4511	APP_ERROR_DATALINKINITIALIZE_FAILED	IEC104 datalink Layer initialization failed
-4512	APP_ERROR_INVALID_FRAMESTART	IEC104 receive Invalid start frame 68
-4513	APP_ERROR_INVALID_SFRAMEFORMAT	IEC104 receive Invalid s format frame
-4514	APP_ERROR_T3T1_FAILED	IEC104 T3- T1 time failed
-4515	APP_ERROR_UPDATE_FAILED	IEC104 api update function failed
-4516	APP_ERROR_TIMESTRUCT_INVALID	IEC104 time structure invalid
-4517	APP_ERROR_FRAME_ENCODE_FAILED	IEC104 encode operation invalid
-4518	APP_ERROR_INVALID_FRAME	IEC104 receive frame invalid
-4519	APP_ERROR_WRITE_FAILED	IEC104 api write function invalid
-4520	APP_ERROR_SELECT_FAILED	IEC104 api select function invalid
-4521	APP_ERROR_OPERATE_FAILED	IEC104 api operate function invalid
-4522	APP_ERROR_CANCEL_FAILED	IEC104 api cancel function invalid
-4523	APP_ERROR_READ_FAILED	IEC104 api read function invalid
-4524	APP_ERROR_DECODE_FAILED	IEC104 Decode failed
-4525	APP_ERROR_GETDATATYPEANDSIZE_FAILED	IEC104 api get datatype and datasize function invalid
-4526	APP_ERROR_CLIENTSTATUS_FAILED	IEC104 api get client status failed
-4527	APP_ERROR_FILE_TRANSFER_FAILED	IEC104 File Transfer Failed
-4528	APP_ERROR_LIST_DIRECTORY_FAILED	IEC104 List Directory Failed
-4529	APP_ERROR_GET_OBJECTSTATUS_FAILED	IEC104 api get object status function failed
-4530	APP_ERROR_CLIENT_CHANGESTATE_FAILED	IEC104 api client change the status function failed
-4531	APP_ERROR_PARAMETERACT_FAILED	IEC104 api Parameter act function

4.2 Error Values

Error Value	Description	Detail
0	EV_NONE	Everything is ok
-1	EV_PARAM_NULL	Parameter is NULL
-2	EV_INVALID_VALUE	Parameter is Invalid Value
-3	EV_OUT_OF_RANGE	Parameter is Out of Range
-4	EV_ERRROCODE_NULL	Error Code is NULL
-5	EV_NOT_SUPPORTED	Function Not Supported
-6	EV_MAX_LIMIT	Exceeds Maximum Limit
-7	EV_ALREADY_EXISTED	Already Existed With Same Name
-8	EV_NOT_EXISTED	Not Existed
-100	EV_OUT_OF_MEMORY	Memory NOT Available
-101	EV_OS_FREE_MEMORY	Os Free Memory
-102	EV_MEMORY_SIZE_INVALID	Size of the Memory is invalid
-300	EV_SOCKET_SELECT_TIMEOUT	Function Wait for Activity Time Out
-301	EV_SOCKET_READ_NOT_READY	Socket Read not Ready
-302	EV_SOCKET_CONNECTION_CLOSED	Socket Connection closed
-303	EV_SOCKET_NOT_READY	Socket not ready
-350	EV_LIST_IS_EMPTY	Link List is Empty
-351	EV_NOT_FOUND	Element NOT Found
-400	EV_NOT_END_OF_FILE	Not Reached End Of File
-500	EV_SERIAL_INITIAL	Serial Initial
-501	EV_MESSAGE_INCOMPLETE	Not Fully Transmitted or Received
-550	EV_MESSAGE_NOT_AVILABLE	Message Not Avilable
-600	EV_MAC_ADDRESS_NOT_AVILABLE	MAC Address Not Avilable
-4501	APP_ERRORVALUE_ERRORCODE_IS_NULL	APP Error code is Null
-4502	APP_ERRORVALUE_INVALID_INPUTPARAMETERS	Supplied Parameters are invalid
-4503	APP_ERRORVALUE_INVALID_APPFLAG	Invalid Application Flag
-4504	APP_ERRORVALUE_UPDATECALLBACK_CLIENTONLY	Update Callback used only for client
-4505	APP_ERRORVALUE_NO_MEMORY	Allocation of memory has failed
-4506	APP_ERRORVALUE_INVALID_IEC104OBJECT	Supplied IEC104Object is invalid
-4507	APP_ERRORVALUE_FREE_CALLED_BEFORE_STOP	APP state is running free function called before stop function
-4508	APP_ERRORVALUE_INVALID_STATE	IEC104OBJECT invalid state
-4509	APP_ERRORVALUE_INVALID_DEBUG_OPTION	invalid debug option
-4510	APP_ERRORVALUE_ERRORVALUE_IS_NULL	Error value is null
-4511	APP_ERRORVALUE_INVALID_IEC104PARAMETERS	Supplied parameter are invalid
-4512	APP_ERRORVALUE_SELECTCALLBACK_SERVERONLY	Select callback for server only
-4513	APP_ERRORVALUE_OPERATECALLBACK_SERVERONLY	Operate callback for server only
-4514	APP_ERRORVALUE_CANCELCALLBACK_SERVERONLY	Cancel callback for server only
-4515	APP_ERRORVALUE_READCALLBACK_SERVERONLY	Read callback for server only
-4516	APP_ERRORVALUE_ACTTERMCALLBACK_SERVERONLY	ACTTERM callback for server only

-4517	APP_ERRORVALUE_INVALID_OBJECTNUMBER	Invalid total no of object
-4518	APP_ERRORVALUE_INVALID_COMMONADDRESS	Invalid common address Slave cant use the common address -> global address 65535
-4519	APP_ERRORVALUE_INVALID_K_VALUE	Invalid k value (range 1-65534)
-4520	APP_ERRORVALUE_INVALID_W_VALUE	Invalid w value (range 1-65534)
-4521	APP_ERRORVALUE_INVALID_TIMEOUT	Invalid time out t (range 1-65534)
-4522	APP_ERRORVALUE_INVALID_BUFFERSIZE	Invalid Buffer Size (100 -65
-4523	APP_ERRORVALUE_INVALID_IEC104OBJECTPOINTER	Invalid Object pointer
-4524	APP_ERRORVALUE_INVALID_MAXAPDU_SIZE	Invalid APDU Size (range 41 - 252)
-4525	APP_ERRORVALUE_INVALID_IOA	IOA Value mismatch
-4526	APP_ERRORVALUE_INVALID_CONTROLMODEL_SBOTIMEOUT	Invalid control model or SBOTimeOut for typeids from Monitor STATUS_ONLY TimeOut 0
-4527	APP_ERRORVALUE_INVALID_CYCLICTRANSTIME	measured values cyclic tranmit time 0
-4528	APP_ERRORVALUE_INVALID_TYPEID	Invalid typeid
-4529	APP_ERRORVALUE_INVALID_PORTNUMBER	Invalid Port number
-4530	APP_ERRORVALUE_INVALID_MAXNUMBEROFCONNECTION	invalid max number of connection
-4531	APP_ERRORVALUE_INVALID_IPADDRESS	Invalid ip address
-4532	APP_ERRORVALUE_INVALID_RECONNECTION	VALUE MUST BE 1 TO 10
-4533	APP_ERRORVALUE_INVALID_NUMBEROFOBJECT	Invalid total no of object u16NoofObject 0-100
-4534	APP_ERRORVALUE_INVALID_IOA_ADDRESS	Invalid IOA 1-16777215
-4535	APP_ERRORVALUE_INVALID_RANGE	Invalid IOA Range 1-1000
-4536	APP_ERRORVALUE_104RECEIVEFRAMEFAILED	IEC104 Receive failed
-4537	APP_ERRORVALUE_INVALID_UFRAMEFORMAT	Invalid frame U format
-4538	APP_ERRORVALUE_T3T1_TIMEOUT_FAILED	t3 - t1 timeout failed
-4539	APP_ERRORVALUE_KVALUE_REACHED_T1_TIMEOUT_FAILED	k value reached and t1 timeout
-4540	APP_ERRORVALUE_LASTIFRAME_T1_TIMEOUT_FAILED	t1 timeout
-4541	APP_ERRORVALUE_INVALIDUPDATE_COUNT	Invalid update count
-4542	APP_ERRORVALUE_INVALID_DATAPOINTER	Invalid data pointer
-4543	APP_ERRORVALUE_INVALID_DATATYPE	Invalid data type
-4544	APP_ERRORVALUE_INVALID_DATASIZE	Invalid data size
-4545	APP_ERRORVALUE_UPDATEOBJECT_NOTFOUND	Invalid update object not found
-4546	APP_ERRORVALUE_INVALID_DATETIME_STRUCT	Invalid data & time structure
-4547	APP_ERRORVALUE_INVALID_PULSETIME	Invalid pulse time flag
-4548	APP_ERRORVALUE_INVALID_COT	For commands the COT must be NOTUSED
-4549	APP_ERRORVALUE_INVALID_CA	For commands the COT must be NOTUSED
-4550	APP_ERRORVALUE_CLIENT_NOTCONNECTED	For commands The client is not connected to server
-4551	APP_ERRORVALUE_INVALID_OPERATION_FLAG	invalid operation flag in cancel function
-4552	APP_ERRORVALUE_INVALID_QUALIFIER	invalid qualifier
-4553	APP_ERRORVALUE_COMMAND_TIMEOUT	command timeout

-4554	APP_ERRORVALUE_INVALID_FRAMEFORMAT	Invalid frame format
-4555	APP_ERRORVALUE_INVALID_IFRAMEFORMAT	invalid information frame format
-4556	APP_ERRORVALUE_INVALID_SFRAMENUMBER	invalid s frame number
-4557	APP_ERRORVALUE_INVALID_KPA	invalid Kind of Parameter value
-4558	APP_ERRORVALUE_FILETRANSFER_TIMEOUT	file transfer timeout
-4559	APP_ERRORVALUE_FILE_NOT_READY	file not ready
-4560	APP_ERRORVALUE_SECTION_NOT_READY	Section not ready
-4561	APP_ERRORVALUE_FILE_OPEN_FAILED	File Open Failed
-4562	APP_ERRORVALUE_FILE_CLOSE_FAILED	File Close Failed
-4563	APP_ERRORVALUE_FILE_WRITE_FAILED	File Write Failed
-4564	APP_ERRORVALUE_FILETRANSFER_INTERUPTTED	File Transfer Interrupted
-4565	APP_ERRORVALUE_SECTIONTRANSFER_INTERUPTTED	Section Transfer Interrupted
-4566	APP_ERRORVALUE_FILE_CHECKSUM_FAILED	File Checksum Failed
-4567	APP_ERRORVALUE_SECTION_CHECKSUM_FAILED	Section Checksum Failed
-4568	APP_ERRORVALUE_FILE_NAME_UNEXPECTED	File Name Unexpected
-4569	APP_ERRORVALUE_SECTION_NAME_UNEXPECTED	Section Name Unexpected
-4570	APP_ERRORVALUE_INVALID_QRP	INVALID Qualifier of Reset Process command
-4571	APP_ERRORVALUE_DIRECTORYCALLBACK_CLIENTONLY	Directory Callback used only for client
-4572	APP_ERRORVALUE_INVALID_BACKGROUNDSCANTIME	BACKGROUND scan time 0
-4573	APP_ERRORVALUE_INVALID_FILETRANSFER_PARAMETER	Server loadconfig
-4574	APP_ERRORVALUE_INVALID_CONNECTION_MODE	Client loadconfig
-4575	APP_ERRORVALUE_FILETRANSFER_DISABLED	Client loadconfig setings
-4576	APP_ERRORVALUE_INVALID_INTIAL_DATABASE_QUALITYFLAG	Server loadconfig intial databse quality flag invalid
-4577	APP_ERRORVALUE_INVALID_STATUSCALLBACK	Invalid status callback
-4578	APP_ERRORVALUE_DEMO_EXPIRED	Demo software expired contact - support@freyrscada.com}
-4579	APP_ERRORVALUE_SERVER_DISABLED	Server functionality disabled in the api, please contact support@freyrscada.com
-4580	APP_ERRORVALUE_CLIENT_DISABLED	Client functionality disabled in the api, please contact support@freyrscada.com
-4581	APP_ERRORVALUE_DEMO_INVALID_POINTCOUNT	Demo software - Total Number of Points exceeded, maximum 100 points
-4582	APP_ERRORVALUE_INVALID_COT_SIZE	Invalid cause of transmission (COT)size